

The Machine, Part 5: Jagala Serve Karna

August 28, 2026

THE MACHINE

PART 5 OF 5: जगाला SERVE करणं

पाच PARTS चा नकाशा

PART 1	पैसा आणि MACHINE	झाला
PART 2	MACHINE शी बोलणं	झाला
PART 3	MACHINES चं जाळं	झाला
PART 4	DATA आणि आठवणी	झाला
[तुम्ही इथे आहात : शेवटचा part]		
PART 5	जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

आधी Part 4 ची परीक्षा

पाच प्रश्न, Part 4 मधले. आधी स्वतः उत्तर द्या, मग खाली बघा.

1. Excel आणि database मध्ये खरा फरक काय? (6.1)
2. 10 million नोंदींमधून शोध ~23 पावलांत कसा? (6.2)
3. "वजा झाले पण जमा नाही" ही अवस्था का येऊ शकत नाही? (6.4)
4. "दाखवणं शिळं चालतं, बदलणं ताजंच लागतं" चा अर्थ? (6.6)
5. Backup आणि replication मध्ये फरक काय? (6.7)

उत्तर: 1. Excel वही आहे, database दरबान: अनेक हात, crores नोंदी, अवघड प्रश्न, अर्धवट लिहिणं: चारही ठिकाणी वही तुटते. 2. लावलेली यादी (index) + प्रत्येक पावलाला अर्ध फेकणं (शब्दकोशाची युक्ती). 3. कारण transaction: दोन्ही नोंदी एक गट्टा; अर्धा झाला तर database स्वतः उलटा फिरवतो (rollback). 4. Score/किंमत दाखवणं cache मधून चालतं (काही seconds जुनं चालेल); पण stock वजा करणं, पैसे हलवणं: थेट database, कुंडीसह. 5. Backup = काल-रात्रीची काळ-प्रत, चुकीपासून वाचवते; replication = जिवंत प्रत, machine मरण्यापासून वाचवते पण चूकही copy करते. दोन्ही लागतात.

या part मध्ये काय आहे

तुमच्याकडे आता पूर्ण machine आहे: recipe चालते (Part 1-2), जग जोडलेलं आहे (Part 3), आठवण पक्की आहे (Part 4). हा part शेवटचे दोन दरवाजे उघडतो:

Section 7: हे सगळं लाखो लोकांसाठी, एकाच वेळेस, परवडणाऱ्या खर्चात कसं चालतं: scale, cloud, वेग, आणि bill.

Section 8: सगळं जोडून: एक खरं product उघडून त्यात पूर्ण पुस्तक बघणं, काय बनवणं सोपं-अवघड, spec कसा लिहायचा, गोष्टी कशा तुटतात, कोण जिंकतं-मरतं, आणि शेवटी Book 1 ची परीक्षा.

हे तुमच्या business decision मध्ये कुठे येईल: हा पूर्ण part च business आहे. इथे technology आणि पैसा एकाच टेबलावर बसतात: server चं bill, ग्राहकाचा वेग, पडणं-सावरणं, आणि “मी काय बनवू, काय विकत घेऊ, काय टाळू” हे founder चे रोजचे निर्णय.

SECTION 7: खूप लोकांना SERVE करणं

नऊ chapters. एका user पासून crores पर्यंत, आणि त्याच्या bill पर्यंत.

हे तुमच्या business decision मध्ये कुठे येईल: “Viral झालो आणि app पडलं” ही startup ची सगळ्यात महाग गोष्ट आहे: संधी दारात आली आणि दार तुटलं. आणि उलटी बाजू: “चाललं, पण bill ने मारलं.” हा section दोन्हीपासून वाचवतो: गर्दी झेलणं, कधीही बंद न पडणं, वेगवान राहणं, आणि हे सगळं करताना खर्चाचा meter वाचता येणं.

Chapter 7.1 [SPINE]: एका user पासून लाखांपर्यंत

तुमचं app चाललं. 100 users: सगळं छान. एका दिवशी कुठल्या celebrity ने उल्लेख केला, आणि संध्याकाळपर्यंत 100,000 लोक दारात. Server धापा टाकतो, app “loading...” वर अडकतं, नवे users वैतागून जातात. हा क्षण प्रत्येक startup च्या स्वप्नात आणि दुःस्वप्नात एकाच वेळेस असतो.

आता design तुम्ही करा. गर्दी दहापट झाली; रस्ते किती?

पहिला रस्ता: मोठी machine घ्या. जास्त CPU, जास्त RAM. नाव: **vertical scaling** (उभं वाढणं). हलवायाची उपमा: दिवाळीत मागणी दहापट झाली; हलवाई स्वतः दहापट वेगाने काम करायचा प्रयत्न करतो. सोपं आहे (काहीच बदलावं लागत नाही), पण दोन भिंती आहेत: एक, जगातली सगळ्यात मोठी machine ही **एक** मर्यादा आहे; तिच्यापुढे रस्ता बंद. दोन, दुप्पट ताकदीची machine दुप्पट नाही, चौपट महाग असते. आणि तिसरी, सगळ्यात वाईट: ती **एकच** आहे; ती मेली तर सगळं मेलं.

दुसरा रस्ता: अनेक machines लावा. दहा सामान्य servers, प्रत्येक गर्दीचा दहावा हिस्सा घेतो. नाव: **horizontal scaling** (आडवं वाढणं). हलवाई आता दहा counters उघडतो, प्रत्येकावर एक माणूस. हाच खरा खेळ आहे, कारण भिंत नाही: अजून गर्दी? अजून counters. पण दोन नवे प्रश्न जन्मतात, आणि दोन्हीची उत्तरं तुम्ही आधीच शिकला आहात:

प्रश्न 1: कुठला ग्राहक कुठल्या counter वर? दारात एक वाटप करणारा लागतो: आलेली प्रत्येक विनंती जो मोकळ्या server कडे पाठवतो. नाव: **load balancer**: मिठाई दुकानाचा token देणारा माणूस, ST stand चा “platform 4 वर जा” सांगणारा फळा. साधी कल्पना, पण आता पूर्ण system चं दार हाच आहे.

प्रश्न 2: state कुठे? Part 2 आठवा (Chapter 3.6): process ची state. User ची cart server क्रमांक 3 च्या memory त ठेवली, आणि पुढची विनंती load balancer ने server 7 कडे पाठवली: cart गायब! उत्तर Part 4 ने दिलं आहे: state servers वर ठेवायचीच नाही; ती database/cache मध्ये (सामायिक जागी) ठेवायची, आणि servers **रिकामे** (stateless) ठेवायचे: कुठलाही server, कुठलीही विनंती, उत्तर तेच. Counters वरचे कारागीर बदलले तरी चालतं, कारण order ची वही एकाच गळ्यावर आहे.

हे खऱ्या जगात रोज दिसतं: IRCTC tatkal च्या क्षणी, Hotstar world cup च्या दिवशी हजारो servers उभे केले जातात; match संपला की कमी. आणि जिथे हे जमलं नाही तिथे काय होतं तेही तुम्ही बघितलं आहे: concert ची tickets, exam चे forms: “site पडली.”

इथे लोक काय चुकीचं समजतात

”मोठा server घेतला की प्रश्न मिटला.” Vertical scaling हे वेदना- शामक आहे, इलाज नाही: भिंत जवळ येते आणि किंमत उडते. आणि उलटी चूक: पहिल्या दिवसापासून “crores users साठी” बांधत बसणं: ते Chapter 4.5 चं उलटं पाप आहे: users 50 आणि architecture NASA चं. शहाणा क्रम: सुरुवात सोपी (एक server ही चालेल), पण **stateless ठेवा**, म्हणजे गर्दी आली की आडवं वाढणं फक्त “अजून counters” इतकं सोपं राहतं. तयारी design मध्ये; खर्च गरजेच्या वेळी.

MAP वर

रुपयाचा रस्ता: scale झेलता येणं हे startup चं जगणं-मरणं आहे: “viral झालो आणि पडलो” म्हणजे दारात आलेली संधी परत गेली, आणि संधी क्वचित दोनदा दार वाजवते. म्हणून गुंतवणूकदार विचारात: “हे दहापट users झेलेल का?” आणि म्हणूनच cloud (Chapter 7.5) जिंकलं: “अजून counters” एका click वर मिळू लागले. Level 3 च्या या एका कौशल्याने Level 2 ची कुठली स्वप्नं जगतात-मरतात हे ठरतं.

स्वतः बघा (5 मिनिटं)

पुढच्या मोठ्या sale च्या दिवशी (Big Billion Days / Prime Day) किंवा result च्या दिवशी लक्ष द्या: apps थोडी हळी, कधी रांग, कधी “try again”. आता तुम्हाला का ते दिसतं: मागे कुठेतरी counters कमी पडतायत, load balancer रांगा लावतोय. आणि जेव्हा सगळं गुळगुळीत चालतं, तेव्हा आता तुम्हाला माहीत आहे: ते आपोआप नाही; कोणीतरी हे chapter जगून दाखवलं आहे.

विचार करा

1. (derivation) हलवायाकडे रोजची 100 ताटं जातात; दिवाळीत 1,000 लागणार. दोन रस्ते: (अ) दहापट मोठी कढई, (ब) दहा सामान्य कढया + वाटप करणारा. कुठला निवडाल आणि तुमच्या निवडीत “state” कुठे आहे?

उत्तर: (ब). मोठ्या कढईची भिंत लवकर येते (एवढी मोठी कढई बनतच नाही), ती महाग, आणि ती फुटली तर दिवाळी गेली. दहा कढया: भिंत नाही, एक बिघडली तरी नऊ चालू. आणि state?

Recipe आणि हिशोब. जर प्रत्येक कढईवाल्याची स्वतःची recipe आणि स्वतःची उधार-वही असेल, तर दहा counters म्हणजे दहा वेगळी दुकानं: भेळ. Recipe एक (लिहिलेली, सगळ्यांना समान: Part 2!) आणि गल्ला/वही एक (सामायिक database: Part 4!), कारागीर रिकामे (stateless): मगच दहा कढया एक दुकान राहतं. Scale चं पूर्ण रहस्य या एका वाक्यात आहे: काम वाटा, आठवण एकत्र ठेवा.

Chapter 7.2 [SPINE]: कधीही बंद न होणारी machine

Part 4 चं सत्य आठवा: disks मरतात, machines मरतात, **नक्की** मरतात. आणि तरी UPI रात्री 3 वाजताही चालतो, Google कधी “आज बंद आहे” म्हणत नाही. दोन्ही गोष्टी एकत्र कशा खऱ्या असू शकतात?

उत्तराची पहिली ओळ मनात कोरून घ्या: **मोठ्या systems मध्ये machines मरत नाहीत असं नाही; मरणं गृहीत धरलेलं असतं.** Design चा प्रश्न “मेली तर?” नाही; “मेली तेव्हा ग्राहकाला कळू नये” हा आहे.

तीन हत्यारं, तिन्ही तुमच्या ओळखीची:

प्रत्येक भागाच्या प्रती (redundancy). दोन servers जिथे एक पुरला असता; database ची जिवंत प्रत (Part 4 चं replication); दोन वेगळे रस्ते (Part 3: packets नवा रस्ता घेतात). दवाखान्याचा generator: वीज असते तेव्हा तो रिकामा उभा असतो, आणि तो “वाया” जात नाही; तो विमा आहे. विमानाला दोन engines असतात ती स्पर्धेसाठी नाहीत.

मरण ओळखणं (health checks). दर काही seconds नी प्रत्येक server ला विचारलं जातं: “जिवंत आहेस?” उत्तर थांबलं की तो रांगेतून काढला जातो. Load balancer (मागचा chapter) हेच करत असतो: ग्राहक फक्त जिवंत counters कडेच पाठवले जातात.

आपोआप वळवणं (failover). मुख्य मेला की प्रत **आपोआप** पुढे येते: माणसाची वाट न बघता, seconds मध्ये. दुकानाचा भाऊ गल्ल्यावर बसतो; ग्राहकाला कळतही नाही मालक आजारी आहे.

आता उपलब्धतेची मोजपट्टी शिका, कारण ती **विकली-विकत घेतली** जाते. भाषा “nines” ची आहे: वर्षभरात system किती टक्के वेळ चालू होतं?

99%	(two nines)	वर्षात ~3.7 दिवस बंद
99.9%	(three nines)	वर्षात ~8.8 तास बंद
99.99%	(four nines)	वर्षात ~53 मिनिटं बंद
99.999%	(five nines)	वर्षात ~5 मिनिटं बंद

आणि नियम: **प्रत्येक पुढचा nine साधारण दहापट महाग.** कारण प्रत्येक पायरीला अजून प्रती, अजून जागा (दुसरं गाव: building जळते!), अजून आपोआप-वळवण्याची यंत्रणा, अजून सराव.

शेवटचा तुकडा: बदल. Software रोज बदलतं (Part 2), आणि बहुतेक मरणं **बदलानंतर** येतात. मग नव version चढवताना दुकान बंद? नाही: दहा counters आहेत ना (मागचा chapter): एक-एक करून बदला; नऊ चालू, एक नवा होतोय; नवा बिघडला तर लगेच जुन्यावर परत. दुकान उघडं ठेवून रंगकाम.

इथे लोक काय चुकीचं समजतात

”मोठ्या companies कडे जादुई, कधी-न-बिघडणारे servers असतात.” उलटं: त्यांच्याकडे **रोज** शेकडो machines मरतात, आणि कोणाला पत्ता लागत नाही; हीच कला आहे. आणि founder ची चूक: “आपण छोटे आहोत, आपल्याला हे नको.” अर्थ खरं: five nines नको (दहापट खर्च आठवा); पण “database ची प्रत आहे का, आणि ती **पुढे येते** का हे तपासलंय का” हा प्रश्न पहिल्या ग्राहकापासून लागू आहे. Part 4 ची ओळ इथेही: न तपासलेला failover म्हणजे failover नाहीच.

MAP वर

उपलब्धता ही थेट विक्रीची गोष्ट आहे: cloud companies करारात लिहून देतात “इतके nines देऊ, कमी पडलो तर पैसे परत” (या कराराचं नाव **SLA**). आणि उलट्या बाजूने मोजा: Amazon साठी एक minute बंद = lakhs चं नुकसान; UPI साठी एक तास बंद = राष्ट्रीय बातमी. म्हणून सूत्रात: जिथे बंद पडण्याची किंमत प्रचंड, तिथे nines विकत घेतले जातात, आणि ते **बांधता** येणारे engineers सगळ्यात महागड्यांमध्ये आहेत (scarcity: हे chapter वाचणं सोपं; पहाटे 3 ला जगवणं वेगळं).

स्वतः बघा (5 मिनिटं)

आठवा: तुमचं bank app किंवा UPI शेवटचं कधी “down” होतं? क्वचित, आणि झालं तरी minutes. आता तुमच्या घरची वीज आठवा: महिन्यात किती वेळा जाते? तुमच्या खिशातली digital-पैशाची यंत्रणा तुमच्या भिंतीतल्या विजेपेक्षा जास्त भरवशाची झाली आहे: आणि ती जादूने नाही, या chapter च्या तीन हत्यारांनी. (आणि आता downdetector.com माहीत करून घ्या: जगात कोण-कोण आता पडलंय याची जिवंत यादी. मोठे पण पडतात; रोज.)

विचार करा

1. (derivation) तुमच्या नव्या business साठी hosting निवडताय: 99.9% वाला plan स्वस्त आहे, 99.99% वाला लक्षणीय महाग. निर्णय **कशावरून** कराल? (Hint: Part 4 च्या backup-वारंवारतेचा निर्णय आठवा.)

उत्तर: तिथल्यासारखाच हा business प्रश्न आहे, tech नाही: “एक तास बंद पडण्याची **माझ्या** धंद्याला किंमत काय?” Blog/माहिती-site: 8 तास/वर्ष बंद कोणाला कळणारही नाही: 99.9% पुरे, वाचलेले पैसे marketing ला द्या. Payment/appointment घेणारा धंदा: बंद तास = गेलेले ग्राहक + गेलेला विश्वास: महाग plan स्वस्त पडतो. आणि एक गोम: सगळी साखळी मोजा: तुमचं app 99.99% पण त्यात जोडलेली SMS सेवा 99% असेल, तर ग्राहकाला दिसणारी उपलब्धता साखळीतल्या **कमजोर** दुव्याची असते. Nines ची खरेदी नेहमी साखळीने, दुव्याने नाही.

Chapter 7.3 [SPINE]: दिसणारा भाग, न दिसणारा भाग

Restaurant मध्ये दोन जगं असतात: **dining hall** (टेबलं, मेनू, सजावट: ग्राहक जे बघतो) आणि **kitchen** (चुली, साठा, हिशोब: जिथे खरं काम होतं). दोन्हींची कामं वेगळी, माणसं वेगळी, नियम वेगळे. ग्राहक kitchen मध्ये जात नाही; waiter दोन्हींना जोडतो.

Software चं **नेमकं** असंच विभाजन आहे, आणि हे दोन शब्द तुम्ही आता रोज ऐकाल:

Frontend = dining hall. जे तुमच्या **हातातल्या** machine वर चालतं: screens, buttons, रंग, animations. App चं रूप. हे client वर चालतं (Part 3 ची भाषा), म्हणजे **ग्राहकाच्या** phone/browser वर.

Backend = kitchen. जे **server** वर चालतं: नियम, हिशोब, database शी बोलणं, पैसे हलवणं. ग्राहकाला कधीच थेट दिसत नाही; फक्त त्याचे निकाल दिसतात.

विभाजन का? कारण दोघांचे **स्वभाव** वेगळे आहेत. Frontend वारंवार बदलतं (नवा रंग, नवी रचना: रोज प्रयोग); backend सांभाळून बदलतं (पैशाचे नियम रोज बदलत नाहीत: Part 2 ची compiler/interpreter-निवड आठवा: बदलाचा वेग vs चुकीची किंमत). दुकानाचं शोकेस रोज सजवता; तिजोरीची खोली सहसा हलवत नाही.

आणि आता या chapter ची सगळ्यात महत्त्वाची ओळ; ती security ची पहिली वीट आहे: **frontend वर कधीही विश्वास ठेवायचा नाही**. का? Frontend ग्राहकाच्या machine वर चालतं, आणि ग्राहकाच्या machine वर **त्याचं** राज्य आहे: तिथलं काहीही बदलता येतं. Part 1 आठवा: app म्हणजे numbers; आणि आपल्याच phone वरचे numbers चलाख माणूस बदलू शकतो.

म्हणजे: app ने दाखवली किंमत 500, चलाख माणसाने आपल्या बाजूला ती 5 करून “order” पाठवली, तर? **चांगलं backend** म्हणतं: मला काय फरक पडतो तू काय दाखवत होतास; **माझ्या** वहीत किंमत 500 आहे, मी 500 चीच नोंद करणार. किंमत, शिल्लक, परवानगी, नियम: प्रत्येक तपासणी kitchen मध्ये **पुन्हा** होते. Waiter ने कितीही सांगितलं “याने पैसे दिलेत,” गल्ला स्वतः मोजतो. Frontend ची तपासणी सोयीसाठी असते (ग्राहकाला लगेच “चूक” दाखवायला); **खरी** तपासणी backend वरच.

इथे लोक काय चुकीचं समजतात

दोन उलट्या चुका. पहिली: “App सुंदर दिसतंय, म्हणजे product तयार.” Chapter 4.5 चा “90% चा भ्रम” इथेच जन्मतो: dining hall सजला; kitchen अजून रिकामं आहे. दुसरी: “frontend म्हणजे हलकं काम.” नाही: हजार प्रकारच्या screens वर, थरथरत्या network वर, अधीर माणसाच्या बोटांखाली गुळगुळीत चालणं ही स्वतःची खोल विद्या आहे. कामं **वेगळी** आहेत; लहान-मोठी नाहीत. (Hiring च्या भाषेत हे शब्द: frontend developer, backend developer, आणि दोन्ही करणारा full-stack.)

MAP वर

रुपयाचा रस्ता: teams याच रेषेने वाटलेली असतात, पगारही. आणि founder साठी एक नकाशा-खूण: frontend च्या जगात हत्यारं फुकट आणि AI प्रचंड वेगवान झालाय: दिसणारं बनवणं **स्वस्त** होतंय (Chapter 0.4 ची आठवण: जे सगळ्यांना जमतं ते स्वस्त होतं). Scarcity मागे सरकते आहे: बरोबर, सुरक्षित, न पडणारं kitchen बांधणाऱ्यांकडे. दिसणाऱ्याची किंमत घसरते; न दिसणाऱ्याची चढते. हे वाक्य या पुस्तकाच्या पहिल्या part पासून चालत आलं आहे, आणि इथे ते hiring-निर्णय बनतं.

स्वतः बघा (5 मिनिटं)

कुठल्याही app मध्ये order/booking करताना जाणीवपूर्वक बघा: किंमत **कुठे-कुठे** दिसते? यादीत, cart मध्ये, आणि शेवटच्या “confirm” वर. शेवटचा आकडा नेहमी एक क्षण उशिरा येतो: ते backend **पुन्हा** मोजतंय; frontend च्या आकड्यावर त्याचा विश्वास नाही. आणि कधी “किंमत बदलली आहे, पुन्हा तपासा” असा संदेश आला असेल: तो हाच chapter जिवंत: वहीने waiter ला दुरुस्त केलं.

विचार करा

1. (derivation) एका शाळेने exam चं app बनवलं; वेळ वाचवायला उत्तरांची तपासणी **app मध्येच** (frontend वर) ठेवली: प्रश्न, चार पर्याय, आणि बरोबर उत्तरही app च्या आतच. काय होईल, आणि दुरुस्ती काय?

उत्तर: बरोबर उत्तर app च्या आत म्हणजे ते विद्यार्थ्यांच्या **machine वर** आहे: numbers उघडून वाचता येतात (Part 1: सगळं numbers च आहे). कोणीतरी ती काढेल आणि वाटेल; परीक्षाच संपली. दुरुस्ती: प्रश्न frontend वर, पण विद्यार्थ्यांचं उत्तर server ला जाणार आणि **तपासणी kitchen मध्ये**: बरोबर उत्तरं फक्त backend च्या वहीत. नियम एका ओळीत: **जे गुपित आहे किंवा जे पैसे/गुण ठरवतं, ते ग्राहकाच्या machine वर कधीच पाठवायचं नाही.** हा नियम मोडून जगात रोज कुठेतरी परीक्षा, sale, आणि game लुटली जाते.

Chapter 7.4 [SPINE]: दोन programs आपसात बोलतात कसे

Zomato ला delivery वाल्याचा रस्ता नकाशावर दाखवायचा आहे. नकाशा बनवणं म्हणजे उपग्रह, गाड्या, जगाचं survey: decades चं काम. Zomato ते करणार? अर्थात नाही: **Google Maps कडून मागणार**. पण थांबा: मागणार **कसं**? Google च्या office ला फोन? माणसासाठी Google ने screen बनवली आहे; पण इथे माणूस नाही, Zomato चा **program** दुसऱ्या company च्या **program** शी बोलणार आहे.

Part 3 मधून उत्तराचा सांगाडा तुमच्याकडे आहेच: दोन machines बोलतात तेव्हा **आधीच ठरलेले नियम** लागतात (protocol). आता तीच कल्पना एक मजला वर न्या: दोन **programs** बोलतात तेव्हा ठरलेले **प्रश्न आणि उत्तरांचे साचे** लागतात:

”असा प्रश्न विचारा: ‘रस्ता दे: इथून-तिथे.’ उत्तर असं येईल: वळणांची यादी, अंतर, वेळ. चुकीचा पत्ता दिलात तर असा नकार येईल.”

अशा ठरलेल्या प्रश्नोत्तर-साच्यांच्या यादीचं नाव **API**. हॉटेलचं **menu card** हीच कल्पना आहे: हे-हे मागता येतं (यादी), असं मागायचं (क्रमांक सांगा), हे मिळेल (ताट), आणि जे menu त नाही ते मागता येत नाही: kitchen त घुसायची परवानगी नाही, गरजही नाही. API म्हणजे program चं menu card: **आतलं न दाखवता, ठरलेल्या सेवा देणारा दरवाजा**. (Chapter 0.4 चं abstraction, त्याच्या सगळ्यात व्यापारी रूपात.)

तुम्ही रोज दहा APIs मधून जाता, न कळत: app मध्ये “Login with Google” (तुमच्या app ने Google चा API वापरला), weather app (हवामान खात्याचा API), प्रत्येक UPI app (NPCI चा API: Part 3 मध्ये protocol म्हणून भेटला; apps साठी तो API च आहे), payment चं Razorpay-पान (दुकानाने पैसे घेण्याचं पूर्ण kitchen भाड्याने घेतलं: फक्त API जोडला).

आणि इथे एक करार-धर्म आहे: API हा **शब्द दिलेला** असतो. हजार apps तुमच्या API वर बांधलेली आहेत; तुम्ही साचा बदललात (प्रश्नाचं रूप, उत्तराचं रूप), तर ती हजार apps **एका क्षणात** तुटतात. म्हणून जुना साचा चालू ठेवून नवा शेजारी उघडतात (versioning: menu बदललं तरी जुन्या ग्राहकांची थाळी बंद करत नाहीत, आधी सांगून मगच).

इथे लोक काय चुकीचं समजतात

”API म्हणजे engineers चा कुठलातरी बारीक technical शब्द.” नाही: API हा **धंद्याचा दरवाजा** आहे. API देणं म्हणजे जाहीर करणं: “माझ्या दुकानावर तुम्ही **तुमची** दुकानं बांधा.” आणि API **वापरणं** म्हणजे दुसऱ्याच्या जमिनीवर भिंत बांधणं: सोय प्रचंड, पण जमीनमालक भाडं वाढवू शकतो. हे खरोखर घडलं: Google Maps ने 2018 मध्ये API चे भाव अनेक पटींनी वाढवले; रातोरात हजारो apps चं महिन्याचं bill उडालं, काही धंदे बदलावे लागले. दुसऱ्याच्या API वर उभा असलेला business हा **भाडेकरू** आहे; भाडेकरूने करार वाचावा.

MAP वर

रुपयाचा रस्ता: “API विकणं” ही स्वतः एक अर्थव्यवस्था आहे. Razorpay/ Stripe म्हणजे “पैसे घेण्याचं kitchen, API ने भाड्याने”: प्रत्येक व्यवहारावर हिस्सा. Twilio: SMS चं kitchen. Google Maps: नकाशाचं. आणि आता AI: OpenAI/Claude चा API म्हणजे “विचार करण्याचं kitchen, भाड्याने” (Book 2). तुमच्या 4-level नकाशावर बघा: API हे Level 3 च्या आतल्या दुकानांचं **वाणिज्य-रूप** आहे: जो एक कठीण गोष्ट सोपा दरवाजा लावून विकतो, तो हजारो धंद्यांचा पुरवठादार होतो: लोणी सगळे बनवत नाहीत; डेअरीकडून घेतात.

स्वतः बघा (5 मिनिटं)

तुमच्या कुठल्याही app मध्ये हे शोधा: “Login with Google” चं बटण, हवामानाचा तुकडा, नकाशाचा तुकडा, payment चं पान जे क्षणभर वेगळं दिसतं (Razorpay/PayU चा रंग). प्रत्येक तुकडा म्हणजे **दुसऱ्या** company चं kitchen, API ने जोडलेलं. एक app मोजा: आत किती companies चे APIs दिसतात? पाच-सात सहज निघतील. आजचं software म्हणजे स्वतःचं थोडं + APIs ची जोडणी.

विचार करा

1. (derivation) तुमचा business आहे: शहरातल्या 200 tiffin-सेवांना ग्राहक मिळवून देणारं app. दोन प्रश्न: (अ) तुम्ही कुठले APIs **वापराल** (विकत घ्याल)? (ब) पुढे-मागे तुम्ही कुठला API **द्याल** (विकाल)?

उत्तर: (अ) पैसे: Razorpay-सारखा (स्वतःचं payment-kitchen बांधणं म्हणजे वेडेपणा); नकाशा/ अंतर: Maps चा; SMS/WhatsApp सूचना: Twilio-सारखा; Login: Google चा. तुमची खरी मेहनत फक्त **तुमच्या** वेगळेपणावर: tiffin-सेवांची यादी, subscriptions, जुळवणी. (ब) मोठे झालात की तुमच्याकडे एक दुर्मिळ गोष्ट जमते: शहरातल्या tiffin-सेवांचा जिवंत data (कोण, कुठे, किती जागा शिल्लक). “Corporate canteen apps ना tiffin-पुरवठा जोडून देणारा API” हा तुमचा माल होऊ शकतो: तुम्ही भाडेकरूचे जमीनमालक होता. धडा: सुरुवातीला APIs **घेऊन** वेळ वाचवा; मोठेपणी API **देऊन** platform व्हा. (Chapter 3.5 चा जकात-नाका, आता तुमच्या बाजूने.)

Chapter 7.5 [SPINE]: विकत घेण्याऐवजी भाड्याने

2000 साली startup सुरू करायची म्हणजे: servers **विकत** घ्या (lakhs चे), त्यांच्यासाठी थंड खोली, वीज-backup, आणि ते सांभाळणारी माणसं. Users येण्याआधी हा खर्च; आणि अंदाज चुकला तर दोन्ही बाजूंनी मार: कमी घेतले तर गर्दीत पडलात, जास्त घेतले तर रिकामे servers पैसे खातायत.

मग एक गोष्ट घडली जिने हे सगळं बदललं. Amazon (दुकान!) ने स्वतःसाठी प्रचंड machine-यंत्रणा बांधली होती; दिवाळी-गर्दीसाठी बांधलेली क्षमता एरवी **रिकामी** असायची. कोणाला तरी सुचलं: ही रिकामी क्षमता **भाड्याने** दिली तर? 2006 मध्ये त्यांनी दार उघडलं, आणि आज ते दुकान (AWS) मूळ दुकानापेक्षा जास्त नफा कमावतं.

या भाड्याच्या जगाचं नाव **cloud**. Part 3 मध्ये वचन दिलं होतं ते आज फेडू: **cloud म्हणजे हवा नाही; cloud म्हणजे दुसऱ्याच्या building मधल्या लाखो machines, ज्या तुम्ही minutes मध्ये, click वर, भाड्याने घेता, आणि वापराल तेवढंच bill भरता.** मोठे तीन मालक: AWS (Amazon), Azure (Microsoft), Google Cloud.

भाडं का जिंकलं, ते तुमच्या रोजच्या शहाणपणातच आहे: लग्नासाठी hall **भाड्याने** घेता, विकत नाही; रोजच्या प्रवासाला गाडी विकत घेता, पण एका trip साठी Ola. नियम: **गरज स्थिर असेल तर विकत; गरज वर-खाली होणारी असेल तर भाड्याने.** आणि startup ची गरज म्हणजे शुद्ध वर-खाली: आज 100 users, उद्या (कदाचित) 100,000, परवा (कदाचित) परत 500. Cloud ने तीन गोष्टी दिल्या:

आधी पैसे नकोत. Servers विकत घ्यायचा डोंगर-खर्च गेला; भाडं वापराबरोबर वाढतं. (हिशोबाच्या भाषेत: CapEx चं OpEx झालं: एकरकमी गुंतवणुकीचं रूपांतर मासिक खर्चात.)

लवचीकपणा. Chapter 7.1 चे “अजून counters” आता एक click आहेत: match च्या दिवशी हजार servers, रात्री परत शंभर. ह्याचं नाव **elastic**: रबरासारखं ताणणं-आकसणं.

जग दारात. Mumbai, Singapore, Frankfurt: प्रत्येक खंडात त्यांची buildings (**regions**) तयार आहेत. तुमचं app जगभर पोहोचवायला जहाज नको; पुढच्या chapter चा विषय.

पण भाड्याचं जग गुलाबी नाही; दोन काटे लक्षात ठेवा. एक: भाडं **कधीच थांबत नाही**, आणि वाढत्या वापराबरोबर bill असा वाढतो की एक दिवस “स्वतःचं घेतलं असतं तर स्वस्त पडलं असतं” हा हिशोब खरा होतो: Dropbox सारख्या मोठ्या, **स्थिर** वापराच्या companies खरंच cloud सोडून स्वतःच्या

machines वर परतल्या आणि crores वाचवले. दोन: **lock-in**: एका मालकाच्या खास सेवांवर खोल बांधलं की दुसरीकडे जाणं महाग होतं; भाडेकरूला घर बदलणं सोपं नसतं (Chapter 7.4 चा जमीनमालक-धडा, इथे पूर्ण इमारतीच्या आकारात).

इथे लोक काय चुकीचं समजतात

”Cloud स्वस्त असतं.” नेहमी नाही. Cloud **लवचीक** असतं; तेच त्याचं खरं मोल. छोट्या/वर-खाली गरजेला ते स्वस्त **पडतं**; मोठ्या स्थिर गरजेला महाग. आणि दुसरी चूक: “cloud मध्ये टाकलं की सुरक्षित/सांभाळलेलं.” मालक building, वीज, machine सांभाळतो; तुमचा data, तुमचे passwords, तुमचा backup (Part 4!) ही **तुमचीच** जबाबदारी राहते. भाड्याच्या घरात मालक भिंती देतो; कुलूप तुम्हीच लावायचं.

MAP वर

रुपयाचा रस्ता: cloud हा platform-जकातीचा (Chapter 3.5) सगळ्यात भव्य अवतार आहे: जगातल्या **लाखो** businesses चं महिन्याचं भाडं तीन मालकांकडे वाहतं; AWS एकटा वर्षाला ~100 billion dollars. तुमच्या 4-level नकाशावर: Level 3 चं हे तळमजल्याचं दुकान इतकं मोठं झालं की देश जागे झाले: “आपला data आपल्या देशात राहावा” (data localization) या नियमांनी India त cloud-buildings चा पाऊस पडतोय. Level 3 चा निर्णय Level-सरकार पर्यंत: pattern ओळखीचा आहे (Part 3, Chapter 5.8).

स्वतः बघा (5 मिनिटं)

तुम्ही स्वतः cloud चे भाडेकरू आहात: Google Photos/Drive चं महिन्याचं 130 रुपयांचं भाडं (Chapter 2.1) म्हणजे Google च्या building मधल्या disks वरची तुमची खोली. आणि एक शोध करा: search करा “AWS pricing calculator” आणि उघडून फक्त **बघा**: CPU तासाला, GB महिन्याला, data बाहेर GB ला: सगळ्याचे दर लिहिलेले आहेत, भाजी-मंडईच्या फळ्यासारखे. पुढच्या-पुढच्या chapter मध्ये (7.8) हेच दर वाचून bill समजायला शिकू.

विचार करा

1. (derivation) दोन businesses: (अ) exam-निकांलांची site: वर्षातले 360 दिवस जवळजवळ शून्य traffic, निकालाच्या 5 दिवशी देशभर स्फोट. (ब) कारखान्याचं आतलं hisheb-software: रोज त्याच 200 कर्मचार्यांचा सारखा वापर, वर्षभर. कोणी भाड्याने, कोणी विकत, आणि का?

उत्तर: (अ) Cloud चा जन्मच जणू यासाठी: 360 दिवस जवळजवळ शून्य भाडं, स्फोटाच्या दिवशी हजार machines, नंतर परत शून्य. विकत घेणं म्हणजे वर्षभर रिकाम्या राक्षसाला खाऊ घालणं. (ब) गरज स्थिर आणि माहीत: इथे “विकत” (किंवा साधं एक server भाड्याने, पण lock-in शिवाय) स्वस्त पडू शकतं; लवचीकपणाची गरजच नाही, मग त्याचं premium का भरा? नियम एका ओळीत: **वर-खाली होणारी गरज = भाड्याचं premium फायद्याचं; सपाट गरज = premium वाया.** आणि खरा शहाणपणा: दर 6 महिन्यांनी हा हिशोब पुन्हा करणं, कारण business वाढला की उत्तर बदलतं (Dropbox ची गोष्ट).

Chapter 7.6 [DEPTH]: सगळे एकदम आले तर काय तुटतं

(DEPTH chapter. पण तुमच्या आयुष्यात कधीही tatkal, flash sale, निकालाचा दिवस किंवा viral क्षण येणार असेल, तर हा तुमच्या कामाचा आहे.)

Chapter 7.1 ने शिकवलं: गर्दी आली की counters वाढवा. मग प्रश्न संपला? नाही. एक जुनी म्हण नव्या रूपात: **साखळीची ताकद तिच्या सगळ्यात कमजोर दुव्याइतकीच**. लग्नाची जेवणाची रांग आठवा: ताटं भरपूर, वाढपी भरपूर, पण **पोळ्यांचा** counter एकच: सगळी रांग तिथेच अडते. Counters दहा केले, पण सगळे एकाच database ला विचारतात (Part 4): database अडला, तर दहा रिकामे counters फक्त सुंदर दिसतात.

या अडणाऱ्या जागेचं नाव **bottleneck** (बाटलीची मान: बाटली केवढीही मोठी, ओतण्याचा वेग मान ठरवते). आणि गर्दीच्या दिवसाची पहिली विद्या: **अंदाजाने नाही, मोजून** bottleneck शोधणं. मोठ्या systems मध्ये प्रत्येक भागावर मीटर लावलेले असतात (या मोजमापांचं नाव **metrics**): कुठला भाग किती भरलाय, कुठे रांग वाढतेय. Founder च्या भाषेत: dashboard. मीटरशिवाय गर्दीचा दिवस म्हणजे अंधारात आग विझवणं.

Bottleneck सापडला; आता हत्यारं, क्रमाने:

- 1. रुंद करा.** पोळ्यांचे counters वाढवा: database ची प्रत (Part 4 चं replication वाचनासाठी), अजून servers. सरळ, पण नेहमी शक्य नसतं (पैशाची वही एकच असते: Chapter 6.4).
- 2. रांग लावा.** Part 4 चं queue-हत्यार: सगळं आत्ताच नको (SMS नंतर!). आणि ग्राहकालाही **दिसणारी** रांग: BookMyShow ची “You are in queue” ची पायरी म्हणजे design, अपयश नाही: आत गोंधळ होण्यापेक्षा दारावर शिस्त.
- 3. दार आवळा.** मंदिरात दर्शनाला एका वेळेस मोजकेच सोडतात; आत चेंगराचेंगरी होण्यापेक्षा बाहेर रांग बरी. Systems मध्ये याचं नाव **rate limit**: “एका user ला minute ला इतक्याच विनंत्या.” चलाख programs (bots) आणि वेडी गर्दी दोन्हींपासून हेच वाचवतं.

आणि आता ती गोष्ट जी गर्दीच्या दिवसाला खरी भयानक बनवते: **cascade** (कोसळत जाणं). साखळी बघा: एक भाग **हळा** झाला (मेला नाही, हळा!) -> त्याची वाट बघणारे तिष्ठले -> त्यांनी कंटाळून **पुन्हा** प्रयत्न केला (retry) -> म्हणजे गर्दी दुप्पट -> हळा भाग अजून हळा -> अजून retries... एकाच्या खोकल्याने पूर्ण रांगेला ताप. जगातले मोठे outages बहुतेक असेच: कोणी मारलं नाही; system स्वतःच्या retries खाली चिरडली. इलाजाचं नाव मजेदार आहे: **circuit breaker** (घरातल्या MCB वरून): एखादा

भाग सतत अपयशी दिसला, तर त्याला काही क्षण **विचारणं थंबवा**; त्याला श्वास द्या, ग्राहकाला “थोड्या वेळाने” सांगा. आजारी माणसाला दर मिनिटाला “बरा झालास?” विचारत राहणं हाच त्याचा आजार वाढवतं.

इथे लोक काय चुकीचं समजतात

”Traffic दुप्पट होणार, servers दुप्पट केले, तयारी झाली.” Bottleneck **हलतो**: servers दुप्पट केले की अडचण database वर जाते; ती सोडवली की payment-API वर (Chapter 7.4 चा भाडेकरू: त्यांचंही एक rate limit आहे!); ती सोडवली की SMS वर. गर्दीची तयारी म्हणजे एक आकडा दुप्पट करणं नाही; **पूर्ण साखळी** चालवून बघणं. मोठ्या companies निकालाच्या/sale च्या आधी नकली गर्दी घालून सराव करतात (load test): लग्नाआधीची पंगत-rehearsal.

MAP वर

रुपयाचा रस्ता: गर्दीचा क्षण हाच धंद्याचा **सुगीचा** क्षण असतो: sale चा तास, निकालाचा दिवस, match ची रात्र. तिथे पडणं म्हणजे वर्षाची कमाई त्याच दाराशी सोडणं; म्हणून मोठ्या e-commerce companies सण-sale आधी महिने फक्त हीच तयारी करतात. आणि उलटं: “आमची site sale च्या दिवशीही पडत नाही” ही ओळ थेट जाहिरातीत विकली जाते: भरवसा हाच माल (Part 3 च्या encryption-धड्याची आठवण: विश्वास बाजार उभा करतो).

स्वतः बघा (5 मिनिटं)

पुढच्या वेळेस कुठल्या app वर “You are in queue” किंवा “Please try after some time” दिसलं, तर आता ते **वाचा**: पहिलं म्हणजे दारावरची शिस्त-रांग (हत्यार 2), दुसरं बहुधा rate limit किंवा circuit breaker (हत्यार 3, किंवा आजारी भागाला दिलेला श्वास). आधी हे “errors” वाटायचे; आता design दिसतंय. आणि सगळ्यात गोड: आता तुम्हाला कळतं की हे संदेश **नसणारी** आणि सरळ कोसळणारी site कुठल्या पायरीवर आहे.

विचार करा

1. (derivation) तुमच्या नव्या course-वेबसाईटवर उद्या सकाळी 10 ला admission उघडणार: 50,000 जागांसाठी अंदाजे 500,000 लोक येतील. वरची तीन हत्यारं **तुमच्या** case मध्ये नेमकी कशी लावाल? आणि एक खास प्रश्न: जागा-वाटप कुठल्या part च्या कुठल्या हत्याराने चालेल?

उत्तर: दार आवळा: 10 च्या ठोक्याला सगळ्यांना आत न सोडता दारावर दिसणारी रांग (queue page): आत मोजके, बाकी क्रमांकासह वाट बघत. रुंद करा: फक्त-वाचण्याची पानं (माहिती, फॉर्मचं रूप) आधीच cache/CDN वर (Chapter 6.6, पुढचा chapter): ती गर्दी मूळ system ला शिवूच नये. रांग लावा: भरलेले फॉर्म queue त; पावतीं लगेच (“अर्ज मिळाला, क्रमांक हा”), प्रक्रिया मागे सावकाश. आणि जागा-वाटप: ती **मोजकी वस्तू** आहे: Chapter 6.4 चं lock + transaction, database च्या वहीवर; तिथे मात्र रांग नशिबावर नाही, कुंडीने. चार parts ची हत्यारं एका सकाळसाठी: पुस्तक असं एकत्र येतं.

Chapter 7.7 [SPINE]: गोष्टी हळू का वाटतात

तुम्ही app उघडलं आणि पूर्ण एक second काहीच झालं नाही. “फक्त” एक second: पण Google-Amazon च्या मोजमापांनी (Chapter 2.3) शिकवलंय की हा second थेट पैसा आहे. आज तो second उघडून बघू: तो जातो कुठे?

एक विनंती धरा: तुम्ही दूरच्या (America तल्या) एका site चं पान मागितलं. घड्याळ लावा:

नाव -> number (DNS, 5.5)	~30 ms
"बोलूया?" ची सुरुवातीची देवाण-घेवाण	~200 ms
(नमस्कार + कुलूप-करार, 5.4/5.6: India-America चे 2-3 फेरे!)	
तुमची विनंती जाऊन उत्तर येणं	~150 ms
server चा विचार (DB, 6.2)	~100 ms
पान मोठं आहे: photos वगैरे येणं	~400 ms
screen वर रंगवणं (frontend, 7.3)	~100 ms

	~1 second

आता सगळ्यात महत्त्वाची गोष्ट **दिसते**: server ने फक्त 100 ms खाल्ले. उरलेला जवळजवळ सगळा वेळ **रस्त्यावर** गेला: फेऱ्यांमध्ये. आणि प्रत्येक फेरीची किंमत **अंतराने** ठरते (Part 1 चं भौतिक सत्य: प्रकाशाचा वेगही मर्यादा आहे; India-America फेरा ~200 ms च्या खाली जाऊच शकत नाही). म्हणून वेगाची दोन मोठी हत्यारं:

हत्यार 1: अंतर मारा. उत्तर जर Pune तूनच मिळालं तर फेरा 200 ms चा 20 ms होतो. म्हणजे प्रती **जगभर** ठेवा: Chapter 6.6 चा cache, आता जागतिक आकारात. या धंद्याचं नाव **CDN** (Cloudflare, Akamai): जगभरच्या 300+ शहरांमध्ये त्यांची छोटी godowns; तुमच्या site चे photos, videos, न-बदलणारी पानं तिथे आधीच ठेवलेली; ग्राहकाला जवळच्या godown मधून मिळतं. Hotstar ची match crores लोक बघतात ती अशीच: प्रत्येक frame Mumbai-Delhi-Chennai च्या godowns मधून; मूळ server पर्यंत कोणीच जात नाही. Amul चं दूध प्रत्येक गावात पहाटे पोहोचलेलं असतं; कोणी Anand ला जात नाही.

हत्यार 2: फेऱ्या मारा. जे अंतर मारता येत नाही (तुमचा खरा हिशोब मूळ server वरच होणार), तिथे **फेऱ्यांची संख्या** कमी करा: एका फेरीत जास्त सामान (दहा प्रश्नांऐवजी एक एकत्र प्रश्न), लागणारं आधीच मागवून ठेवणं, दुसऱ्या पानाचं सामान पहिल्या पानासोबतच. मंडईची यादी करून **एकाच** फेरीत सगळं आणणारा cook (2.3) विरुद्ध प्रत्येक कांद्यासाठी वेगळी फेरी मारणारा.

आणि एक तिसरं, चोरटं हत्यार: **वाटणं**. ग्राहकाची घड्याळ त्याच्या **डोक्यात** असते. पान येईपर्यंत रिकामा पांढरा screen विरुद्ध लगेच दिसणारा सांगाडा + “होतंय” ची हालचाल: दुसरं दुप्पट वेगवान **वाटतं**, वेळ तोच असूनही. Restaurant मध्ये लगेच पाणी-पापड आल्यावर जेवणाची वाट सुसह्य होते. हे फसवणूक नाही; हा माणसाच्या अनुभवाचा design आहे: पण फक्त खऱ्या वेगाच्या **जोडीला**, बदली म्हणून नाही.

इथे लोक काय चुकीचं समजतात

”App हळू आहे: मोठा/वेगवान server घ्या.” वरचा हिशोब पुन्हा बघा: server चे 100 ms दुप्पट वेगवान केले तर second चा फायदा 50 ms: कोणाला कळणारही नाही. खरे खाणारे रस्त्यावर आहेत: अंतर, फेरे, आणि पानाचं **वजन** (Chapter 2.1: photo = हजारपट text; अर्ध जड पान = अर्धा वेळ). म्हणून वेगाचं काम **मोजण्याने** सुरू होतं (कुठे जातोय second?), अंदाजाने नाही. Chapter 7.6 चा धडाच: मीटरशिवाय इलाज नाही.

MAP वर

रुपयाचा रस्ता: CDN ही स्वतः एक industry आहे (Cloudflare: billions revenue), आणि तिचं मूळ हेच: **अंतर हे पैशात मोजता येतं**. आणि तुमच्या India-first नकाशासाठी खास ओळ: तुमचा ग्राहक 4G वर, स्वस्त phone वर, गावात आहे: त्याच्यासाठी पानाचं वजन आणि फेरे शहरी WiFi पेक्षा दसपट महाग आहेत. जो founder “माझ्या office च्या WiFi वर तर झकास चालतं” वर थांबतो (Chapter 4.4 चा माहोल-धडा!), तो आपला खरा बाजार हरवतो. India त जिंकलेली apps (WhatsApp, UPI apps) हलक्या वजनासाठी **वेड्या** होत्या, हा योगायोग नाही.

स्वतः बघा (5 मिनिटं)

दोन प्रयोग: (1) एखादी परदेशी, कमी-प्रसिद्ध site उघडा आणि मग कुठलीही मोठी Indian site: फरक **जाणवेल**: तो अंतराचा + CDN चा आहे. (2) Phone च्या settings मधून थोडा वेळ “data saver” लावून नेहमीची apps वापरा: कुठली app तरीही चपळ राहते ते बघा: तिच्या team ने हा chapter जगलेला आहे.

विचार करा

- (derivation) तुमचं tiffin-app (7.4 वालं) गावांमध्ये न्यायचंय. वेगासाठी तीन बदल सुचवा: एक CDN-वाला, एक फेऱ्यावाला, एक वजनावाला. (आणि एक traps-प्रश्न: order **देण्याची** क्रिया CDN वर ठेवता येईल का?)

उत्तर: CDN: menu चे photos, app चं न-बदलणारं सामान जवळच्या godown मधून (एकदा ठेवलं की crores वेळा फुकट-जवळ). फेऱ्या: app उघडताच लागणारं सगळं (आजचा menu + शिल्लक subscription + पत्ता) **एका** एकत्र उत्तरात; दहा वेगळे प्रश्न नकोत. वजन: photos दाबलेले (Chapter 2.1 ची WhatsApp-युक्ती), पहिलं पान photo-शिवाय ही वाचता येईल असं. आणि trap: **नाही**. Order देणं म्हणजे बदल घडवणं: मोजकी वस्तू, ताजी वही, transaction (6.4, 6.6 चा नियम: “दाखवणं शिळं चालतं, बदलणं ताजंच लागतं”). CDN फक्त **दाखवण्याच्या** प्रती वाटतो; **बदल** नेहमी मूळ वहीकडेच जातो. दोन्हींची गल्लत केली की जुना menu आणि दुहेरी orders: दोन्ही chapters एकत्र समजले याचीच ही परीक्षा होती.

Chapter 7.8 [SPINE]: हे सगळं चालवायचा खर्च किती

Part 1 पासून एक गाणं वाजतंय: software ची **copy** फुकट (Chapter 1.6), म्हणून leverage चा राजा. आज त्या गाण्याची दुसरी ओळ, जी पहिल्या यशानंतर प्रत्येक founder ला रात्री जागवते: copy फुकट आहे, पण **चालवणं** फुकट नाही. दर महिन्याला cloud चं bill येतं, आणि ते वाचता न येणं म्हणजे अंधारात सही करणं (Section 2 च्या opener ची भविष्यवाणी).

चला bill उघडू. कितीही लांब असलं तरी त्याच्या **मोठ्या तीन** ओळी याच असतात, आणि तिन्ही तुम्ही या पुस्तकात शिकला आहात:

1. Compute: विचाराचं भाडं. CPU/RAM किती वेळ वापरले (Chapter 3.6: process चालू = meter चालू). दर: machine च्या आकाराप्रमाणे, **तासाला**. हे भाडं खातं कोण: प्रत्येक विनंतीमागचा हिशोब, आणि (Book 2 ची चाहूल) AI चालवायचं असेल तर खास GPU-machines: त्यांचे दर सामान्यांच्या दहा-वीसपट.

2. Storage: आठवणीचं भाडं. किती GB, किती महिने (Chapter 2.1: जे मोजलं जातं तेच विकलं जातं). दर: **GB-महिन्याला**. स्वस्त वाटतं: आणि आहेही: पण data फक्त **वाढतो** (कोण delete करतं?), म्हणून ही ओळ दर महिन्याला थोडी जाड होत जाते: हळू वाढणारं वजन.

3. बाहेर जाणारा data: दरवाजाची जकात. आणि हा तिसरा, सगळ्यात **कपटी**: आत येणारा data बहुधा फुकट; तुमच्या servers मधून जगाकडे **बाहेर** जाणाऱ्या प्रत्येक GB ला दर (नाव: **egress**). Text ला काही वाटत नाही; पण Chapter 2.1 चं प्रमाण आठवा: video = लाखपट. प्रत्येक user ने बघितलेला प्रत्येक video = बाहेर गेलेले MBs = जकात. म्हणून video-app चा egress हाच त्याचा राक्षस, आणि म्हणूनच CDN (7.7) नुसता वेगाचा नाही, **पैशाचा** सौदा आहे: एकदा godown त ठेवा, CDN च्या स्वस्त दराने वाटा.

आता या तीन ओळी धंद्याच्या भाषेत आणा; याचं नाव **unit economics**: **एका** user मागे महिन्याला खर्च किती, आणि कमाई किती? हलवायला ताटामागचा खर्च पाठ असतो; तुम्हाला user-मागचा हवा. आणि इथे Part 1 चं जुनं भाकीत (Chapter 2.1 चा विचार-प्रश्न) पूर्ण वर्तुळ फिरून परत येतं: text-app चा user-खर्च जवळजवळ शून्य: फुकट users पोसता येतात; video-app चा खर्च खरा आणि जड: पहिल्या दिवसापासून कमाईचा रस्ता लागतो. **Product चा आकार त्याच्या business model चा आकार ठरवतो**: तेव्हा हे वाक्य theory होतं; आता तुम्ही ते bill च्या तीन ओळींमध्ये बोट ठेवून दाखवू शकता.

आणि एक इशारा, अनुभवाने लिहिलेला: **bill shock**. पहिली viral रात्र येते, सकाळी founder आनंदाने dashboard उघडतो: users दहापट! आणि मग mail उघडतो: bill **तीसपट**. का तीस, दहा का नाही? कारण गर्दीत सगळं महाग-रूप घेतं: जादा machines (7.1), जादा egress, आणि कुठलातरी विसरलेला भाग (एक न-दाबलेला video, एक loop मधली query) गर्दीने राक्षस बनवलेला. इलाज तोच जुना: **मीटर आणि कुंपण**: रोजच्या खर्चाचा alert (“दिवसाला X च्या वर गेलं तर मला कळवा”) पहिल्या दिवशी लावायचा: घराच्या विजेचं bill महिन्याने कळतं; cloud चं **रोज** बघता येतं, ही सोय न वापरणं गुन्हा आहे.

इथे लोक काय चुकीचं समजतात

“Software बनलं की चालवणं फुकट: copies फुकट असतात ना!” आता तुम्हाला पूर्ण चित्र आहे: copy (recipe ची) फुकट; **serve** करणं (compute + storage + egress) चालू meter. Software चा leverage खरा आहे, पण तो “खर्च शून्य” नाही, “खर्च **user-मागे छोटा आणि घसरणारा**” असा आहे. आणि दुसरी चूक: bill engineer चं काम समजणं. Bill हा business चा आरसा आहे: कुठलं feature महाग आहे, कुठला user तोट्याचा आहे, हे तिथे लिहिलेलं असतं: founder ने महिन्यातून एकदा स्वतः तीन मोठ्या ओळी वाचणं म्हणजे दुकानदाराने गल्ला स्वतः मोजणं.

MAP वर

रुपयाचा रस्ता, पूर्ण साखळी, शेवटच्या वेळेस: तुमचा user 300 रुपये देतो (Level 1) -> तुमचं app (Level 2) -> cloud चं bill: compute/ storage/egress (Level 3 ची जकात) -> cloud मालक chip-वाल्यांना (TSMC पर्यंत: Part 1 चं वर्तुळ पूर्ण). आणि AI आलं की या साखळीत एक जाड नवी ओळ चढते: GPU चं भाडं: Book 2 चा अर्धा हिशोब तिथेच आहे. जो founder ही साखळी वाचू शकतो, तो किंमत **ठरवू** शकतो; जो वाचू शकत नाही, तो “फुकट देऊ, बघू पुढे” म्हणत बुडतो.

स्वतः बघा (5 मिनिटं)

तुमच्या phone चं गेल्या महिन्याचं data-वापराचं पान उघडा (Settings -> Data usage): कुठल्या app ने किती GB खाल्ले? YouTube/Instagram वर, WhatsApp खाली: तेच 1 : हजार : लाख चं प्रमाण. आता उलट्या बाजूने विचार करा: तुम्ही जे **उतरवलं**, ते कोणाच्या तरी bill मध्ये **egress** म्हणून चढलं आहे. तुमचं रोजचं scroll म्हणजे कोणाच्या तरी meter ची फिरती चकती: आता ती चकती तुम्हाला दिसते.

विचार करा

1. (derivation) तुमच्या app मध्ये दोन features सारखेच लोकप्रिय: (अ) मित्रांशी text-gappa, (ब) 24-तासांनी गायब होणारे video-status. पैसे संपत आलेत; एक feature तात्पुरतं बंद करायचंय. Bill च्या तीन ओळींनी ठरवा: कुठलं, आणि का?

उत्तर: (ब) बंद करा. तीन ओळी वाचा: (अ) text: compute चिमूट, storage चिमूट (Chapter 2.1: 1 KB!), egress चिमूट: लाखो users जवळजवळ फुकट. (ब) video: storage जड (प्रत्येक status MBs मध्ये, रोज नवा), आणि egress राक्षसी: प्रत्येक बघणं = बाहेर जाणारे MBs x बघणाऱ्यांची संख्या. एकच feature, पण user-मागचा खर्च हजारपट. आणि खोल धडा: दोन्ही “सारखीच लोकप्रिय” दिसतात **वापराच्या** आकड्यात; **खर्चाच्या** आकड्यात ती एकाच जातीची नाहीतच. Founder ची नजर म्हणजे प्रत्येक feature ला दोन्ही आकड्यांनी बघणं: प्रेम किती, आणि पोट किती खातं.

Chapter 7.9 [SPINE]: Part 0 चा दावा, तपासलेला

Chapter 0.5 मध्ये आपण एक दावा लिहून ठेवला होता:

दावा: Machine समजल्यावर तुम्ही कुठल्याही business कडे बघून सांगू शकाल की त्यात technology ने पैसा कुठे बनेल, काय बनवणं स्वस्त आहे, काय महाग, आणि AI त्यात कुठे घुसेल.

आज तो दावा **तपासायचा** आहे: कुठल्याही “tech” business वर नाही; मुद्दाम एका साध्या, जुन्या धंद्यावर: **गल्लीतलं किराणा दुकान**. काका एकटे चालवतात; वही, तराजू, उधारी, आणि रोजची डोकेदुखी. चला, पुस्तक घेऊन दुकानात शिरू.

काकांच्या डोकेदुख्या ऐका आणि प्रत्येकीखाली पुस्तकातलं हत्यार लिहा:

”उधारीची वही सांभाळता सांभाळता जीव जातो; कोणाची किती, कधीची, आठवत नाही.” ही Part 4 ची Chapter 6.1 वाली वही आहे: नोंदी वाढल्या, प्रश्न अवघड झाले. हत्यार: छोटा database (आजच्या जगात एक तयार app). आणि “वसुलीची आठवण आपोआप जावो”: Chapter 6.5 ची रांग: आठवडाभराने आपोआप message. पैसा कुठे बनला: बुडणारी उधारी कमी = थेट खिशात.

”महिन्याअखेरीस कुठला माल संपला, कुठला पडून सडला, हे आधी कळत नाही.” हा data आहे जो **आधीच** वहीत आहे; प्रश्न विचारायचं हत्यार नाही (Chapter 6.2 ची query!). रोजची विक्री नोंदली की “काय संपत आलंय” ही यादी आपोआप. आणि एक पाऊल पुढे: गेल्या तीन महिन्यांच्या pattern वरून “पुढच्या आठवड्यात किती लागेल” चा अंदाज: हा Level 4 चा दरवाजा (Book 2), पण त्याची **जमीन** (नोंदलेला data) Level 3 नेच बनते. AI इथे घुसेल: आणि तो **तेव्हाच** घुसू शकेल जेव्हा आधी नोंदी digital झाल्या असतील. (AI ची भूक data आहे; उपाशी AI निरुपयोगी.)

”UPI आल्यापासून पैसे थेट खात्यात येतात, बरं झालं.” झालेलंच आहे: काका आधीच Level 3 चे ग्राहक आहेत (Part 3 चा protocol, Chapter 6.4 चे transactions त्यांच्यासाठी कोणीतरी चालवतंय). खूण: technology धंद्यात **थेंबाथेंबाने** घुसते, आणि सगळ्यात आधी तिथे जिथे दुखणं सगळ्यात तीव्र (पैशाची देवाणघेवाण).

”Online-order वालं app बनवून देतो म्हणतो एकजण, दीड लाख मागतोय.” आता तुमची Part 2 + 5 ची नजर: काय **विचाराल**? Chalta tukda दाखवा (4.5); कुठले APIs वापरणार, स्वतः काय बनवणार (7.4: payment/नकाशा विकत, फक्त जोडणी); चालवायचा खर्च काय (7.8: याची गिन्हाईकं 200; cloud चं bill चिमूटभर असायला हवं); आणि data कुठे, backup काय (6.7). चार प्रश्नांत दीड लाखाची किंमत उघडी पडते: इथे **बनवण्यासारखं** फार नाहीच: तयार viटा जोडायच्या आहेत.

आता मागे सरका आणि बघा काय घडलं: एका दुकानावर पुस्तकाची पाचही parts बोलली. आणि सूत्र (0.2) शेवटचं लावा: काकांचं दुकान: size लहान (एक गल्ली), scarcity कमी (शेजारी दुसरं दुकान), leverage जवळजवळ शून्य (काका झोपले = दुकान बंद). पण **हजार** दुकानांची उधारी-वसुली सोपी करणारं app: size मोठा, आणि leverage पूर्ण (recipe एकदा, हजार दुकानं). म्हणजे पैसा **काकांच्या** दुखण्यात आहे, पण मोठा पैसा ते दुखणं **हजारदा** सोडवण्यात आहे. हीच Level 2 -> Level 3 ची उडी, आतून बघितलेली.

इथे लोक काय चुकीचं समजतात

”आधी technology शिका/बनवा, मग बघू कुठे लावायची.” उलटा क्रम आहे, आणि आता तुमच्याकडे पुरावा आहे: आपण दुकानात **दुखण्यांपासून** सुरुवात केली, आणि प्रत्येक दुखण्याने आपलं हत्यार स्वतः निवडलं. Technology-आधी-माणसं हा क्रम उलटला (चकाचक app, गरज कोणालाच नाही) की Chapter 0.3 ची विहीर आठवा: दोरी सुंदर, विहीर कोरडी. प्रश्न नेहमी तोच: **कोणाचं, कुठलं दुखणं, किती तीव्र?** उत्तर Level 1 मध्ये; हत्यारं Level 3-4 मध्ये.

MAP वर

हा chapter स्वतःच नकाशाची परीक्षा होती. पण एक शेवटची खूण: लक्षात घ्या की काकांच्या दुकानात आपल्याला **नवीन शोध** एकही लागला नाही: सगळी हत्यारं तयार होती (apps, UPI, cloud, APIs): फुकट किंवा स्वस्त (Chapter 3.7 चं वरदान). म्हणजे आजच्या जगात संधीची व्याख्या बदलली आहे: **हत्यारं शोधणं हा खेळ संपला; दुखणं नेमकं ओळखून हत्यारं जोडणं हा खेळ उरला.** आणि त्या खेळात तुमच्याकडे आता दोन्ही डोळे आहेत: दुखण्याचा (Level 1-2) आणि हत्याराचा (Level 3-4).

स्वतः बघा (5 मिनिटं)

हा chapter तुमच्या हातात एक **चौकट** देऊन जातो; ती कागदावर लिहा: (1) या धंद्यात रोज **पुन्हा-पुन्हा** काय होतं? (recipe-संधी, 1.3) (2) **वही** कुठे आहे आणि ती कागदी का digital? (6.1) (3) **मोजकी वस्तू** कुठली? (6.4) (4) **वाट** कुठे बघावी लागते? (6.5) (5) पैसा **कुठून कुठे** वाहतो? (0.3). पाच प्रश्न: कुठल्याही धंद्याचा X-ray.

विचार करा

- (derivation) आता दावा **तुम्ही** तपासा: तुमच्या ओळखीचा एक धंदा निवडा (शेतकरी, डॉक्टरचा दवाखाना, पूजा-साहित्याचं दुकान, tuition class: कुठलाही). वरची पाच-प्रश्नांची चौकट चालवा आणि **एक** जागा शोधा जिथे technology ने सगळ्यात आधी पैसा बनेल.

उत्तर: उत्तर तुमचं-तुमचं येईल; चौकट बरोबर चालली की ते **आपोआप** दिसतं: बहुधा ते वही-दुखणं असेल (नोंदी, उधारी, appointments) किंवा वाट-दुखणं (रांगा, आठवणी, follow-up), कारण तिथे recipe स्पष्ट, data छोटा, आणि तयार हत्यारं स्वस्त. आणि एक तपासणी: तुमचं उत्तर “अमुक app बनवू” ने सुरू होत असेल तर चौकट पुन्हा चालवा; ते “यांचं तमुक दुखणं रोजचं आहे” ने सुरू व्हायला हवं. दावा तपासला: जर हे जमलं, तर Part 0 चं वचन या पुस्तकाने पाळलं आहे: आणि श्रेय पुस्तकाचं नाही, तुमच्या जोडण्यांचं आहे.

SECTION 8: सगळं जोडून

सहा chapters: पुस्तकाचा कळस. एक खरं product आतून-बाहेरून, founder चा सोपं-अवघड नकाशा, spec लिहिण्याची कला, गोष्टी तुटण्याच्या खऱ्या गोष्टी, जिंकण्या-मरण्याचे नियम, आणि शेवटी परीक्षा.

हे तुमच्या business decision मध्ये कुठे येईल: आतापर्यंत पुस्तकाने तुम्हाला **भाग** दिले; हा section **जोडणी** देतो: जी नजर एक product उघडून त्यात सगळे भाग एकाच वेळेस बघते. तीच नजर घेऊन तुम्ही बनवाल, विकत घ्याल, आणि नकार द्याल.

Chapter 8.1 [SPINE]: एक खरं product उघडून बघणं

रात्रीचे आठ वाजले आहेत. तुम्ही Zomato उघडता, biryani शोधता, order करता, पैसे देता, आणि 40 मिनिटांनी जेवता. **60 seconds** चा तुमचा हात-भाग. आता तेच 60 seconds पुन्हा, पण या वेळेस पुस्तकाच्या डोळ्यांनी: प्रत्येक पावलावर कंसात chapter-क्रमांक: हा तुमचा पूर्ण अभ्यासाचा एकत्रित नकाशा आहे.

- 1. तुम्ही icon ला हात लावता.** काचेखालचा sensor, current ची साखळी, billions transistors (1.1). App: storage मधून RAM च्या मेजावर चढणारी recipe; जन्मणारी process (2.5, 3.6). एकच phone हजार machines बनतो: universal machine (1.6).
- 2. App “zomato.com” ला हाक मारतं.** नावाचा number होतो: DNS ची शिडी, cache मुळे बहुधा एका फेरीत (5.5). कुलूप-करार: https, दोन किल्ल्या (5.6). तुमची विनंती तुकड्यांत: packets, प्रत्येक स्वतंत्र, TCP जुळवणारा (5.3). रस्ता: WiFi चे 10 meter, मग Jio ची काच, कदाचित समुद्राखालची तार (5.7).
- 3. विनंती Zomato च्या दारात.** दारावर वाटप करणारा: load balancer, मागे रिकामे (stateless) servers चे counters (7.1). प्रत्येक server: cloud मधली भाड्याची machine (7.5), जिच्यावर लिहिलेली recipe कुठल्या भाषेत होती आणि कशी भाषांतरित झाली हे तुम्हाला माहीत आहे (3.1-3.3).
- 4. Kitchen विचार करतं.** तुमच्या भागातली restaurants: database ला query, index मुळे क्षणात (6.1, 6.2). Menu आणि photos: cache आणि CDN मधून, जवळच्या godown मधून (6.6, 7.7): म्हणून यादी इतकी पटकन येते. Photos चं वजन: pixels चे numbers, दाबलेले (2.1, 2.2).
- 5. तुम्ही “Place Order” दाबता.** आता खेळ बदलतो: दाखवणं संपलं, बदलणं सुरू (6.6 चा नियम). किंमत frontend वर दिसली होती; खरी तपासणी kitchen मध्ये (7.3). पैसे: Razorpay चा API (7.4), मागे UPI चा protocol (5.4), आणि दोन banks च्या वह्यांमध्ये एक गट्टा: transaction, वजा-जमा एकत्र किंवा काहीच नाही (6.4).
- 6. हिरवी खूण.** आणि त्याच क्षणी चिठ्ठ्या रांगेत: restaurant च्या screen ला, delivery शोधणाऱ्याला, तुमच्या SMS ला (6.5). तुम्हाला वाट लागली नाही; रांगा मागे चालतायत. Delivery वाला नकाशावर सरकतो: Maps चा API (7.4), त्याच्या phone वरून दर काही seconds नी येणारे छोटे packets.

7. आणि पडद्यामागे, सतत. प्रत्येक भागाच्या प्रती जिवंत: failover तयार (7.2). रात्रभर backup चालेल (6.7). मीटर फिरतोय: compute, storage, egress (7.8). आणि पैशाची साखळी वाहते आहे: तुमचे 300 -> restaurant -> Zomato -> cloud/APIs -> chips: Level 1 पासून TSMC पर्यंत (0.3, 7.8).

हे सगळं: **60 seconds मध्ये, प्रत्येक वेळेस, crores लोकांसाठी.** आणि आता या chapter चा खरा मुद्दा: हे **कोणी एकाने** बांधलेलं नाही. Transistor वाल्याला biryani माहीत नाही; Zomato च्या engineer ला silicon माहीत नाही. प्रत्येक थर खालच्या थरावर **विश्वास** ठेवतो आणि फक्त आपलं काम करतो: abstraction (0.4) ही फक्त युक्ती नाही; ती **सहकाराची** पद्धत आहे: लाखो अनोळखी माणसांनी थरा-थरांत वाटून घेतलेलं एक काम. मंडईची भाजी तुमच्या ताटात येते तेव्हाही हेच घडतं: शेतकरी-वाहतूक-मंडई-दुकान-घर: कोणी एक “जेवण बनवत” नाही. Machine चं जगही तसंच पिकतं.

इथे लोक काय चुकीचं समजतात

“एवढं सगळं एका order मागे? मग हे बांधणं अशक्यच.” उलट निष्कर्ष काढा: हे सगळं **तयार** आहे: थर बांधलेले आहेत, दारं (APIs) उघडी आहेत, भाडं परवडणारं आहे. नवा founder 60 seconds ची ही साखळी **पहिल्या दिवशी** वापरू शकतो; त्याला ती **बांधावी** लागत नाही. अशक्य वाटणारं चित्र हीच तुमची सगळ्यात मोठी सोय आहे: तुमचं काम फक्त वरचा, तुमचा, वेगळा थर.

MAP वर

हा chapter च नकाशा आहे; पण एक शेवटचं बोट: वरच्या 7 पावलांमध्ये **किती companies** कमावल्या ते मोजा: telecom, cloud, CDN, payment, Maps, database-सेवा, chips... एक biryani, पंधरा दुकानं. Level 3 म्हणजे एक company नाही; ती एक **बाजारपेठ** आहे, आणि प्रत्येक थराच्या फटीत एक धंदा मावतो. तुमची फट कुठली?

स्वतः बघा (5 मिनिटं)

आज कुठलंही **एक** app वापरताना (order, ticket, payment: काहीही) मनातल्या मनात ही 7 पावलं चालवा: हात-DNS-दार-kitchen-बदल-रांगा- मीटर. पहिल्यांदा हळू जाईल; तिसऱ्या वेळेस आपोआप दिसेल. ज्या दिवशी app “आपोआप उघडताना” तुम्हाला थर दिसायला लागतील, त्या दिवशी हे पुस्तक तुमच्या डोक्यातून बोटांत उतरलं.

विचार करा

1. (derivation) आता तीच trace **तुम्ही** लिहा: UPI ने मित्राला 500 पाठवण्याची. कमीत कमी सहा पावलं, प्रत्येकावर chapter-क्रमांक. (मग खालचं उत्तर बघा.)

उत्तर (सांगाडा): (1) बोट-PIN: switches, आणि PIN कधीच उघडा प्रवास करत नाही: encryption (1.1, 5.6). (2) app -> तुमच्या bank चं दार: DNS, https, packets (5.5, 5.6, 5.3). (3) NPCI चा protocol: दोन banks ची भाषा एक (5.4). (4) दोन वहांमध्ये गठ्ठा: lock + transaction, वजा-जमा एकत्र (6.4). (5) हिरवी खूण तुम्हाला; SMS/नोंदी रांगेतून (6.5). (6) मागे: प्रती, failover, backup (7.2, 6.7); मीटर (7.8); आणि यंत्रणा 99.99% जागी (7.2): म्हणून पहाटे तीनला चहावाल्याकडेही हे चालतं. सहा पावलांत अर्ध पुस्तक: आणि आता ते **तुम्ही** लिहिलंत, वाचलं नाहीत.

Chapter 8.2 [SPINE]: काय बनवणं सोपं, काय अवघड

Founder ची सगळ्यात महाग चूक कुठली माहीत आहे? Feature ची किंमत न ओळखणं: सोपं समजून अवघडात उडी, किंवा अवघड समजून सोप्याला नकार. आज तो नकाशा बनवू: पुस्तकातल्या हत्यारांनीच.

सोपं (आठवडे): जे तयार थरांची फक्त जोडणी आहे. Forms, याद्या, नोंदी, dashboards: म्हणजे “वही + दाखवणं” (6.1 + 7.3). Login, payment, नकाशा, SMS: विकत मिळतं (7.4 चे APIs). असं product म्हणजे लग्नाचा मांडव: बांबू, कापड, खुर्च्या सगळं भाड्याने; कारागिरी फक्त **जोडणीत**. AI ने (Book 2) ही जोडणी अजून वेगवान केली आहे: सोपं आता **अजून** सोपं आहे.

मध्यम (महिने): जिथे तुमचा स्वतःचा थर बांधावा लागतो. दोन बाजूंची बाजारपेठ (ग्राहक + दुकानदार दोघांना पटवणं), नाजूक जुळवाजुळव (delivery वाल्यांचं वेळापत्रक), offline-आधी design (गावाकडचं network: 7.7). इथे तयार तुकडे **पुरत नाहीत**; इथे तुमची खरी मेहनत आणि तुमचा खरा **किल्ला** (वेगळेपण) दोन्ही असतात.

अवघड (वर्ष, फौज): जिथे भौतिकशास्त्र आणि मोजक्या वस्तूंची शर्यत एकत्र येतात. Crores लोकांसाठी **एकाच क्षणी** बरोबर असणं: tatkal-स्केलचं seat-वाटप (6.4 x 7.6), स्वतःचं payment-network (5.4 च्या protocol चं स्वतःचं रूप), realtime video crores ना (2.1 चं वजन x 7.8 चं egress), स्वतःचे AI models शून्यातून. इथे “बनवू का?” चा उत्तर बहुधा “विकत घ्या/भाड्याने घ्या” असतं: दूध लागतं म्हणून कोणी म्हैस पाळत नाही.

आणि आता या नकाशाची **मुख्य फसवणूक**, जी प्रत्येक नव्या founder ला एकदा तरी लुटते: **दिसणं आणि असणं यांचा संबंध उलटा आहे.** “WhatsApp सारखं app” ऐकायला सोपं: दिसतं साधं ना! पण आत: crores च्या रांगा (6.5), end-to-end किल्ल्या (5.6), 99.99% जागेपण (7.2): जगातल्या सगळ्यात अवघड यादीत. उलट “दुकानाची उधार-वही app” ऐकायला किरकोळ, पण खरोखर आठवड्यांचं काम आणि खरा पैसा (7.9 चे काका). **Demo सोपा, production अवघड** (4.5 चा 90%-भ्रम); आणि “थोडंसं वेगळं हवं” ही ओळ ऐकली की कान टवकारा: platform च्या वाटेवरून उतरलात की किंमत दहापट (4.1 च्या जोड्या).

इथे लोक काय चुकीचं समजतात

“Spec मधली एक ओळ = कामाची एक ओळ.” आता तुमच्याकडे मोजपट्टी आहे: ओळ वाचून विचारा: ही **जोडणी** आहे (सोपं), **नवा थर** आहे (मध्यम), की **मोजक्या वस्तूंची शर्यत/भौतिकशास्त्र** आहे (अवघड)? “Users एकमेकांना दिसतील” ही सात शब्दांची ओळ: पण ती realtime असेल तर (7.6, 7.7)

महिन्यांचं दार उघडते. Engineer “हे अवघड आहे” म्हणतो तेव्हा तो बहुधा टाळत नाही; तो हीच मोजपट्टी वाचतोय: आता तुम्ही त्याच्याशी **त्याच** भाषेत बोलू शकता.

MAP वर

रुपयाचा रस्ता: सोपं झालं म्हणजे **स्वस्त** झालं म्हणजे तिथे गर्दी आणि स्पर्धा (0.2 चं सूत्र: scarcity घसरली). नुसती जोडणी असलेली products आता AI सुद्धा बनवतो: तिथे किल्ला नाही. किल्ला दोन जागी उरतो: **मध्यम** पट्ट्यात (तुमची खास जुळवाजुळव, तुमचा data: 6.1) आणि **वाटपात** (8.5 मध्ये येतंय). म्हणून धोरण एका ओळीत: सोप्याने **सुरुवात** करा (वेग!), पण खंदक मध्यम पट्ट्यात खणा; अवघडाला API ने हात लावा, फौजेने नाही.

स्वतः बघा (5 मिनिटं)

तुमच्या phone मधली तीन apps घ्या आणि प्रत्येकीला विचारा: हिचा **अवघड** भाग कुठला असेल? Zomato: वेळेवर delivery ची जुळवाजुळव. WhatsApp: crores realtime + किल्ल्या. UPI app: पैशाची शुद्धता. आणि मग गंमत बघा: प्रत्येकीचा अवघड भाग **दिसत** नाही (0.4 चं जुनं सत्य, शेवटच्या वेळेस): screen वर सगळ्या सारख्याच “सोप्या” दिसतात.

विचार करा

1. (derivation) दोन ideas, दोन्ही “booking”: (अ) डॉक्टरांसाठी appointment-वही: रुग्ण वेळ निवडतो, आठवण जाते. (ब) “सगळ्या शाळांचे निकाल एकाच क्षणी, live” दाखवणारी site. कुठली सोपी, कुठली अवघड, आणि **नेमकं का?** (पुस्तकातले chapter-क्रमांक द्या.)

उत्तर: (अ) सोपी-ते-मध्यम: वही + दाखवणं (6.1, 7.3), आठवणी रांगेतून (6.5); मोजकी वस्तू (वेळेची जागा) आहे, पण शर्यत छोटी: एका दवाखान्यात एका क्षणी दोघं-चौघंच भांडतील (6.4 पुरतं). (ब) अवघड: “एकाच क्षणी” + “सगळे” = 7.6 चा राक्षस (निकालाचा दिवस!), वाचनाचा महापूर CDN/cache ने झेलावा लागेल (6.6, 7.7), आणि चुकीचा निकाल दाखवणं परवडत नाही म्हणजे शिळेपणाची मोजपट्टी कडक (6.6). दोन्हीत “booking/निकाल” हे शब्द साधेच; फरक **कोण, किती, एकाच क्षणी का** या तीन प्रश्नांत. Feature च्या शब्दांना नाही, या तीन प्रश्नांना किंमत असते.

Chapter 8.3 [SPINE]: Spec कसा दिसतो

तुम्ही कधीतरी कोणाला तरी (माणसाला किंवा AI ला) म्हणाल: “मला असं app बनवून हवंय.” त्या क्षणी जे कागदावर द्याल, त्याचं नाव **spec** (specification: नेमकं-काय-हवं ची लिहिलेली मागणी). आणि Part 2 पासूनचं सत्य आठवा (3.1): machine ला **exact** लागतं, आणि बनवणाऱ्या माणसालाही. अर्धवट spec = Chapter 4.1 ची फट = “मला हे नको होतं” चा महाग शोध.

घर बांधणाऱ्याला “छान घर बांधा” म्हणत नाही; नकाशा देतो. पण spec म्हणजे 100 पानं नाहीत. चांगला spec **सात प्रश्नांची** उत्तरं आहे: आणि सातही तुम्ही या पुस्तकात शिकला आहात:

- 1. कोणासाठी, कुठलं दुखणं?** (Level 1 पासून सुरुवात: 0.3, 7.9.) एक ओळ: “कोण, रोज काय सहन करतं.”
- 2. मुख्य प्रवास, पावला-पावलाने.** “User हे करतो -> त्याला हे दिसतं -> मग हे.” (1.3 ची recipe-भाषा: तुम्हीच recipe चे पहिले लेखक.)
- 3. मोजकी वस्तू आणि कडक नियम कुठले?** पैसे, जागा, stock: जिथे शर्यत आणि गठ्ठे लागतील (6.4). ही ओळ लिहिली नाहीत, तर ती **चूक होऊन** भेटते.
- 4. कुठला data कायमचा, आणि किती हरवणं चालेल?** वही कुठली (6.1), backup चा भूक-प्रश्न (6.7 चा “किती तास परवडतात?”).
- 5. आत्ता काय, नंतर काय चालेल?** हिरव्या खुणेआधी काय झालंच पाहिजे; रांगेत काय जाऊ शकतं (6.5).
- 6. किती लोक, कुठून, किती वेगात?** शंभर की लाख (7.1); शहर की गाव (7.7); सामान्य दिवस की स्फोटाचे क्षण (7.6).
- 7. काय बनवायचं नाही?** सगळ्यात ताकदवान ओळ. “यात हे **नसेल**” हे लिहिणं म्हणजे 4.1 च्या जोड्यांचा गुणाकार थांबवणं आणि 4.5 चा छोटा तुकडा राखणं.

बघा कसा दिसतो: **टिफिन-सेवेचा spec, नऊ ओळींत:**

कोणासाठी: शहरात एकटे राहणारे; रोजच्या जेवणाची भ्रांत.
 प्रवास: यादीतून सेवा निवड -> महिन्याची वर्गणी -> रोज
 दुपारी डबा; "उद्या नको" रात्री 10 पर्यंत सांगता येतं.
 मोजकं + नियम: सेवेची रोजची डबे-क्षमता (संपली की
 नवीन वर्गणी बंद); पैसे आधी, महिन्याचे.
 Data: वर्गण्या + रोजचे बदल + पैसे; एक तासाहून जास्त
 नोंदी हरवणं चालणार नाही.
 आत्ता vs नंतर: वर्गणी + payment आत्ता; रोजची आठवण,
 सेवेला यादी: रांगेतून.
 किती: पहिल्या वर्षी एक शहर, ~2,000 वर्गणीदार; रोज
 सकाळी 9-11 गर्दीचा फुगा.
 नाही: delivery-मुलांचं app नाही (सेवा स्वतः बघेल),
 रोजची सुट्टी-निवड नाही (फक्त "उद्या नको"), ratings
 नाहीत (व1 मध्ये).

नऊ ओळी: पण बनवणाऱ्याच्या डोक्यातले शंभर प्रश्न मिटले, आणि तुमच्या नकळत तुम्ही database ची
 रूपरेषा (4), queue ची यादी (5), आणि scale ची तयारी (6) लिहून दिलीत: **spec म्हणजे पुस्तकाची
 चौकट, गद्यात.**

इथे लोक काय चुकीचं समजतात

दोन टोकं. "तोंडी सांगितलं, कळेल त्याला": Chapter 3.1 उलटा वाचा: माणूसही अर्धवट ऐकतो, आणि AI
 तर **नेमकं** तेच बनवतो जे लिहिलंय, जे **हवं होतं** ते नाही (1.3: machine निर्णय चालवते). दुसरं टोक:
 "100 पानांचा दस्तऐवज": तो कोणी वाचत नाही, आणि पहिल्या चालत्या तुकड्याने (4.5) तो शिळा होतो.
 Spec **जिवंत** असतो: छोटा, नेमका, प्रत्येक तुकड्यागणिक वाढणारा. आणि सोनेरी नियम: प्रत्येक
 ओळीला विचारा "ही ओळ चुकली तर काय तुटेल?": उत्तर "काही नाही" असेल तर ओळ हटवा.

MAP वर

रुपयाचा रस्ता: spec लिहिता येणं ही **founder ची** सगळ्यात स्वस्त आणि सगळ्यात परताव्याची skill
 झाली आहे: कारण AI च्या जमान्यात बनवणं स्वस्त होतंय (3.2 ची भविष्यवाणी: व्याकरण स्वस्त, साफ
 विचार महाग), म्हणजे **मागणी नेमकी सांगता येणं** हीच उरलेली scarcity. सात प्रश्नांचा हा साचा तुमच्या
 दुसऱ्या मेंदूत (वहीत) ठेवा: पुढच्या प्रत्येक कल्पनेला आधी हे नऊ-ओळी रूप द्या; मगच कोणाशी बोला.
 एक तासाची ही सवय लाखांच्या गैरसमजांची विमा-पॉलिसी आहे.

स्वतः बघा (5 मिनिटं)

वरचा टिफिन-spec झाकून ठेवा आणि **तुमच्या** डोक्यातल्या कुठल्याही एका कल्पनेचा सात-प्रश्नी spec लिहा: नऊ-दहा ओळीच. अडाल तिथे खूण करा: अडलात **म्हणजे** तिथेच तुमच्या कल्पनेतलं धुकं आहे: spec लिहिणं हे बनवणाऱ्यासाठी नाही, आधी **तुमच्यासाठी** असतं: धुकं शोधायचं यंत्र.

विचार करा

1. (derivation) वरच्या टिफिन-spec मध्ये एक महत्वाचं भोक मुद्दाम ठेवलं आहे: एक अशी रोजची घटना जिचं उत्तर spec देत नाही. शोधा. (Hint: Chapter 4.1 चा चष्मा लावा: “नेहमीचा प्रवास” सोडून कडेला काय घडू शकतं?)

उत्तर: असे एकाहून जास्त सापडतील: हीच 4.1 ची शिकवण: पण सगळ्यात टोचणारं: **payment झालं पण डबा आलाच नाही, तर?** (सेवा बंद पडली, डबेवाला आजारी, पाऊस.) पैसे परत? पुढच्या महिन्यात जमा? कोणाची जबाबदारी? हे spec मध्ये नाही: म्हणजे ते **कोणीतरी दुसरं** ठरवणार: बनवणारा engineer, ऐन वेळी, तुमच्या business चा नियम त्याच्या अंदाजाने लिहीत. Edge case म्हणजे तांत्रिक बारकावा नाही; तो **business चा नियम** आहे जो लिहायचा राहिला. Spec चं खरं काम: आनंदाचा प्रवास नाही, **दुखऱ्या फांद्या** आधी लिहिणं.

Chapter 8.4 [DEPTH]: खऱ्या आयुष्यात गोष्टी कशा तुटतात

(DEPTH chapter. पण यातल्या गोष्टी खऱ्या आहेत, आणि founder म्हणून तुमच्या टेबलावर कधीतरी अशीच एक रात्र येईल.)

पुस्तकभर आपण तुटण्याचे तुकडे शिकलो: bugs (4.1), माहोल (4.4), cascade (7.6), मरणान्या disks (6.7). आज तीन **खऱ्या** गोष्टी: जगातल्या सगळ्यात हुशार teams च्या. हेतू टिंगल नाही; हेतू हा: तुटणं **कसं दिसतं** आणि शहाणी माणसं त्यातून **काय** करतात.

गोष्ट 1: स्वतःचंच दार बंद (Facebook, 2021). एका routine बदलात एक चुकीची आज्ञा गेली, आणि Facebook ने जगाच्या नकाशावरून **स्वतःचे पत्ते** पुसले (Part 3 ची भाषा: त्यांचे रस्ते जगाच्या route-वहीतून गायब). WhatsApp, Instagram: 6 तास जग-बंद. आणि कहर: engineers चे **आतले** दरवाजेही त्याच system वर होते: दुरुस्ती करणाऱ्यांना building त शिरता येईना. धडा 1: **बदल हाच सगळ्यात मोठा धोका** (बहुतेक मोठे मृत्यू deploy नंतर येतात: 7.2 चं एक-एक-counter शहाणपण याचसाठी). धडा 2: दुरुस्तीचा रस्ता **वेगळा** ठेवा: आग विझवायची नळी जळणाऱ्या घरातच ठेवू नका.

गोष्ट 2: एक typo, अर्ध internet (Amazon S3, 2017). एक engineer, एक नेहमीची आज्ञा, एक टंकलेखनाची चूक: हेतूपेक्षा **खूप जास्त** servers बंद केले गेले. आणि cascade (7.6): S3 वर जग टेकलेलं होतं, म्हणून हजारो websites एकदम लंगड्या. धडा: चूक **माणसाची** नव्हती; **व्यवस्थेची** होती: इतकी घातक आज्ञा एका typo इतकी सोपी का होती? कुंपण का नव्हतं ("इतकं बंद करायचं? खात्री आहे?")? शहाण्या systems मध्ये मोठ्या विनाशाचे दरवाजे **मुद्दाम जड** केलेले असतात.

गोष्ट 3: Backup होते; restore नव्हतं (GitLab, 2017). रात्रीची गडबड, थकलेला engineer, चुकीच्या server वर delete. ठीक: backup आहे ना? उघडले: पाच जागी प्रती ठेवायची व्यवस्था होती, आणि **पाचही** कुठल्या ना कुठल्या कारणाने अधुन्या/रिकाम्या निघाल्या. 6 तासांचा data कायमचा गेला. Part 4 ची ओळ त्यांच्याच रक्ताने लिहिलेली आहे: **न तपासलेला backup म्हणजे backup नाहीच** (6.7). आणि एक गोष्ट त्यांनी जगाला शिकवली: त्यांनी हे सगळं **जाहीर, live** सांगितलं: लपवलं नाही: आणि जग रागावण्याऐवजी त्यांच्यावर विश्वासाने वाढलं.

तिन्ही गोष्टींतून एकच पद्धत जन्मते, आणि तिचं नाव शिका: **postmortem** (घटनेनंतरचं शवविच्छेदन: पण system चं, माणसाचं नाही). शहाण्या companies प्रत्येक मोठ्या तुटण्यानंतर एक दस्तऐवज लिहितात: काय घडलं (मिनिटा-मिनिटाने), **कुठली व्यवस्था** कमी पडली, आणि पुन्हा होऊ नये म्हणून कुठली **व्यवस्था** बदलली. आणि पहिला नियम: **blameless** (दोषारोप-मुक्त): "कोण चुकलं" विचारायचं नाही; "चूक इतकी **सोपी** का होती" विचारायचं. कारण साधं आहे: माणसाला शिक्षा केली की पुढच्या वेळेस

माणूस चूक **लपवतो**: आणि लपलेली चूक पुन्हा येते, मोठी होऊन. विमान-उद्योग जगात सगळ्यात सुरक्षित याच एका संस्कृतीने: प्रत्येक अपघाताचा जाहीर अभ्यास, दोषारोपाशिवाय; म्हणून प्रत्येक अपघात **शेवटचा** त्या-प्रकारचा.

इथे लोक काय चुकीचं समजतात

”मोठ्या companies कडे असं होत नसेल; आपल्याकडे झालं म्हणजे आपण कच्चे.” वरच्या तीन गोष्टी पुन्हा वाचा: जगातल्या **सगळ्यात** मोठ्या teams च्या आहेत. फरक तुटण्या-न-तुटण्यात नाही; फरक **नंतर** काय होतं यात आहे: लपवणं की शिकणं. आणि founder साठी आरसा: तुमच्या team मध्ये वॉर्ट बातमी तुमच्यापर्यंत **लवकर** येते का उशिरा? उशिरा येत असेल, तर तुमची team कच्ची नाही; तुमची **संस्कृती** महाग आहे: लोक तुम्हाला घाबरून चुका लपवतायत (4.5 ची प्रामाणिक-team खूण).

MAP वर

रुपयाचा रस्ता: विश्वास हा माल आहे (Part 3 पासूनचं गाणं), आणि तुटण्याच्या क्षणी तो **सगळ्यात जास्त** विकला-विकत घेतला जातो. GitLab लपवू शकले असते; त्यांनी उघडलं आणि ग्राहक **वाढले**. उलट ज्या companies तुटणं लपवतात (विशेषतः data-चोरी!), त्या उघडकीस आल्यावर दुप्पट बुडतात: एकदा तुटण्याने, एकदा लपवण्याने. नियम: **तुटणं माफ होतं; लपवणं नाही**. आणि हो: postmortem ची सवय फुकट आहे: पहिल्या दिवसापासून, एका-माणसाच्या company तही.

स्वतः बघा (5 मिनिटं)

तुमच्या स्वतःच्या (कुठल्याही: दुकान, नोकरी, घर) आयुष्यातली एक जुनी, बोचणारी चूक घ्या. तिच्यावर blameless postmortem चे तीन प्रश्न चालवा, कागदावर: (1) नेमकं काय घडलं, क्रमाने? (2) कुठली **व्यवस्था** (सवय, तपासणी, आठवण) असती तर हे टळलं असतं? (3) ती व्यवस्था आज लावता येईल का? “कोण चुकलं” हा प्रश्न यादीत **नाही** हे जाणीवपूर्वक बघा: आणि लक्षात घ्या, या तीन प्रश्नांनी चूक आठवताना टोचणं कमी आणि उपयोग जास्त होतो. हीच पद्धतीची ताकद.

विचार करा

1. (derivation) तुमच्या टिफिन-app (8.3) च्या पहिल्या महिन्यात एक रात्र: चुकीच्या यादीमुळे 200 पैकी 60 जणांना डबे गेलेच नाहीत. Blameless postmortem चा सांगाडा लिहा: घडलं-काय, व्यवस्था-कुठली- कमी, बदल-कुठला. (मग उत्तर बघा.)

उत्तर (सांगाडा): घडलं: रात्री 10 च्या “उद्या नको” बदलांची रांग (6.5) सकाळच्या यादी-निर्मितीनंतर रिकामी झाली: यादी जुनी गेली (6.6 चा शिळेपणा, चुकीच्या जागी!). व्यवस्था कमी: यादी बनल्यावर तिची **तपासणी** नव्हती (एकूण डबे = वर्गणीदार - उद्या-नको: हा साधा ताळा कोणी मोजला नाही: 4.2 ची जाणीवपूर्वक- तपासणी); आणि सेवेकडून “मला 140 च दिसतायत” चा इशारा यायला रस्ताच नव्हता. बदल: यादी बनण्याची वेळ रांग रिकामी झाल्या- **नंतर**; ताळ्याची आपोआप तपासणी, न जुळल्यास माणसाला गजर; आणि प्रत्येक सकाळी सेवेला “आजचे एकूण” ची पावती. बघा: उत्तरात एकही “कोण चुकलं” नाही: तीन नव्या **व्यवस्था** आहेत. हीच सवय, हाच धंदा टिकवते.

Chapter 8.5 [SPINE]: एक technology जिंकते, दुसरी मरते: का?

पुस्तकभर मेलेल्यांची थडगी दिसली: Nokia (1.6), जुनं telephone-जग (5.3), SMS (5.8). आणि प्रत्येक वेळेस एक अस्वस्थ प्रश्न मागे उरला: मेलेले **वाईट** नव्हते. Nokia चे phones दणकट होते; SMS सगळीकडे चालायचा. मग “चांगलं product जिंकतं” हे वाक्य खोटं आहे का?

अर्थ खोटं आहे: आणि तो अर्धा भाग founder साठी सोन्याचा आहे. जिंकण्या-मरण्याचे खरे नियम पाच आहेत; पाचही तुम्ही तुकड्या- तुकड्यांत शिकलात; आज ते एका टेबलावर:

1. जाळ्याची ओढ (network effect). WhatsApp का सोडता येत नाही? App प्रेमासाठी नाही; **माणसं** तिथे आहेत म्हणून. प्रत्येक नवा user जुन्या सगळ्यांसाठी किंमत वाढवतो: n -squared, पण या वेळेस तुमच्या **बाजूने** (5.1 चा राक्षस पाळीव झाला की किल्ला होतो). अशा जागी दुसरा येऊन “10% चांगलं” देऊन जिंकू शकत नाही; त्याला **रिकामं जाळं** विकावं लागतं: आणि रिकाम्या मंडईत कोणी दुकान लावत नाही.

2. दरवाजाचा मालक (platform + default). Chapter 3.5 चा जकात-नाका: जो OS/store/browser चा मालक, तो **default** ठरवतो. आणि माणसं default बदलत नाहीत: जे आधीच चालू आहे ते चालू राहतं. RCS वर्षानुवर्ष Apple च्या दारात उभा होता (5.8): चांगला असून; दार मालकाचं होतं.

3. सोडून जाण्याची किंमत (switching cost). Bank बदलणं, सगळे photos हलवणं, team ला नवं tool शिकवणं: शक्य, पण **वेदना**. जो product तुमचा data, तुमच्या सवयी, तुमच्या जोडण्या धरून ठेवतो, त्याला 10% ने हरवता येत नाही; **10x** लागतो.

4. वेळ (timing). खूप लवकर येणं हे उशिरा येण्याइतकंच घातक: video-calling 2000 साली होतं (network नव्हतं); touch-phones iPhone आधी होते (chips महाग होत्या). तुमची कल्पना नाही, तिच्या **खालचे थर** तयार आहेत का, हा प्रश्न (0.3 चा stack: खालचा मजला कच्चा असेल तर वरचा बांधून मरता). Jio-नंतरच्या India त video- first apps चाललीत हा योगायोग नाही: 2016 ने खालचा मजला ओतला.

5. वाटप (distribution). सगळ्यात कडू आणि सगळ्यात खरा: **ग्राहकापर्यंत पोहोचण्याचा रस्ता** ज्याच्याकडे, तो सामान्य product घेऊनही जिंकतो; रस्ता नसलेला उत्कृष्ट product कपाटात मरतो. म्हणून companies “फुकट” वाटतात (3.7), दरवाजे विकत घेतात (WhatsApp: 19 billion!), आणि म्हणून UPI wallets ना हरवू शकला: प्रत्येक bank, प्रत्येक app हा त्याचा **रस्ता** होता (5.4 चा खुला protocol = हजार रस्ते फुकट).

आता पाचही एकत्र वाचा आणि दिसेल: पाचपैकी **एकही** “product किती छान आहे” बदल नाही. Product चांगलं असावंच लागतं: पण ते **प्रवेश-तिकीट** आहे, विजय नाही. विजय जाळं, दरवाजा, खिळे, वेळ आणि रस्ते यांचा आहे. (आणि आता Nokia चा मृत्यू पुन्हा वाचा: Apple ने चांगला phone नाही, **app-store चं जाळं + दरवाजा** आणला: नियम 1 + 2 एकत्र. आता तुम्हाला बातमी नाही, यंत्र दिसतं.)

इथे लोक काय चुकीचं समजतात

“चांगलं बनवलं की जग दार शोधत येईल.” Founder-जगातली सगळ्यात गोड अफू. आता तुमच्याकडे उतारा आहे: कल्पना सुचली की product-प्रश्नाआधी हे विचारायचं: **माझं जाळं काय? माझा रस्ता काय? माझे खिळे काय? थर तयार आहेत का?** चारही कोरे असतील, तर product कितीही सुंदर असो: तुमच्याकडे अजून **धंदा** नाही, फक्त एक छान वस्तू आहे.

MAP वर

तुमच्या 4-level नकाशाची शेवटची जोडणी: हे पाच नियम म्हणजे Level 2 आणि Level 3 च्या **लग्राचे** नियम आहेत: technology (3) बांधते, पण जिंकवतं ते business चं वाटप-जाळं (2). आणि पुढच्या पुस्तकाची चाहूल: AI मध्ये आत्ता हीच लढाई पेटली आहे: models (product) झपाट्याने सारखे होतायत; लढाई जाळं-दरवाजा-वाटपाची होते आहे (कोणाचा AI कुठल्या phone मध्ये default बसणार?). Book 2 वाचताना हे पाच नियम सोबत ठेवा: बातम्या आपोआप उलगडतील.

स्वतः बघा (5 मिनिटं)

तुम्ही स्वतः एक चालतं-बोलतं उदाहरण आहात: विचारा, मी WhatsApp **का** वापरतो? उत्तरांची यादी करा: “सगळे तिथे आहेत” (नियम 1), “आधीपासून आहे” (नियम 2), “सगळे groups/photos तिथे” (नियम 3). “सगळ्यात सुंदर app आहे म्हणून” हे उत्तर यादीत **नसेल**: आणि हेच या chapter चं जिवंत प्रमाणपत्र.

विचार करा

1. (derivation) Electric गाड्यांची शर्यत चालू आहे. दहा वर्षांनी जिंकलेली company कशाने जिंकलेली असेल: सगळ्यात चांगली **गाडी** बनवून, की दुसऱ्या कशाने? पाच नियम लावा.

उत्तर: गाडी प्रवेश-तिकीट; विजय बहुधा **जाव्याचा** असेल: charging चं जाळं (नियम 1+5: जिथे charger तिथे गाडी खपते, जिथे गाड्या तिथे charger परवडतो: दोन्ही बाजूंनी वाढणारी ओढ), बॅटरी- पुरवठ्याची साखळी आणि किंमत (नियम 4: खालचा थर), service-जाळं आणि software-updates चे खिळे (नियम 3), आणि सरकारी नियम/ subsidies चा दरवाजा (नियम 2 चं सरकारी रूप: 5.8 ची आठवण). म्हणूनच Tesla ने आधी **chargers** पेरले आणि China च्या companies बॅटरी-साखळी धरून बसल्या. गाडी दिसते; युद्ध जाव्यांचं आहे. हे उत्तर कुठल्याही “पुढच्या मोठ्या गोष्टीला” लावता येतं: तीच या chapter ची भेट.

Chapter 8.6 [SPINE]: Book 1 ची परीक्षा

(ही अंतिम परीक्षा नाही: ती Book 2 च्या शेवटी आहे, दोन्ही पुस्तकांवर. ही Book 1 ची तपासणी आहे: machine तुमच्या हातात बसली का?)

नियम सोपे: पुस्तक न उघडता, **बोलून** उत्तर द्या: लिहिणं नको, बडबड चालेल. प्रत्येक प्रश्नाखाली उत्तराचा गाभा आहे: आधी स्वतःचं, मग ताडून बघा. 12 पैकी 9 जमले तर machine तुमची झाली; जे अडले त्यांचे chapter-क्रमांक सोबत: तिथे परत जा, लाज न बाळगता: परत जाणं हाच अभ्यास (front matter चं वचन).

1. पैसा फक्त Level 1 वर का घुसतो? (0.1, 0.3)

गरज फक्त माणसाला असते. Level 2/3/4 आपसात देतात तो पैसा खालून चढत आलेला असतो; साखळी नेहमी एका माणसाच्या गरजेवर संपते.

2. Surgeon पेक्षा app-वाला जास्त का कमावू शकतो? (0.2)

कमाई = size x scarcity x **leverage**; तास सूत्रात नाहीत. Operation प्रत्येक वेळेस हाताने; app एकदा बनते, crores वेळा चालते: copy फुकट.

3. एकच machine camera, bank आणि game कशी होते? (1.6)

Recipe सुद्धा numbers आहेत, म्हणून data च्या जागीच ठेवता येतात: नवी recipe = नवं काम, machine तीच: universal machine.

4. “Machine विचार करते” हे वाक्य कुठे तुटतं? (1.3, 1.5)

आत फक्त आधीच-design केलेल्या gates च्या साखळ्या: प्रत्येक input चं output आधीच ठरलेलं. निर्णय **चालतो**; घेतला माणसाने असतो: recipe लिहिणाऱ्याने.

5. Text : photo : video चं वजन-प्रमाण, आणि ते इतकं का महत्वाचं? (2.1)

1 : हजार : लाख. कारण जिथे वजन तिथे खर्च: storage, egress, वेग: product चा आकारच business model चा आकार ठरवतो.

6. iPhone ची app Android वर का चालत नाही? (3.5)

App machine साठी नाही, **मालकासाठी** (OS) लिहिलेली असते; मालक वेगळा, भाषा वेगळी. आणि मालक असणं = दरवाजा + जकात.

7. मोठ्या software मध्ये bugs नक्की का असतात? (4.1)

परिस्थितींच्या जोड्यांचा स्फोट: जी जोडी लिहिणाऱ्याच्या डोक्यात आली नाही, ती कधीतरी कोणाच्या हाती लागते. मोजणी आहे, हलगर्जीपणा नाही; प्रश्न पकडण्याच्या वेगाचा.

8. Internet चा मालक कोण, आणि तरी नियम कोण बनवत? (5.4, 5.7, 5.8)

मालक कोणीच नाही: हजारो जाळी, समान **नियमांनी** (protocols) जोडलेली. नियम खुल्या संस्था (IETF वगैरे) + मोठ्या companies च्या ताकदीने घडतात; standard बनवणारा बाजार आखतो: UPI/DPI हेच भारताचं उत्तर.

9. कुलूप-icon काय सांगतो, काय नाही? (5.6)

रस्ता सुरक्षित: मधला कोणी वाचू शकत नाही. **समोरचा खरा आहे** हे सांगत नाही: scammer कडेही https असतं. रस्ता तपासला; प्रवासी तुम्हीच तपासायचा.

10. “वजा झाले पण जमा नाही” ही अवस्था का अशक्य आहे? (6.4)

Transaction: दोन्ही नोंदी एक गट्टा: पूर्ण किंवा शून्य; अर्धा झाला तर database स्वतः उलटा फिरवतो (rollback). मोजकी वस्तू + अनेक हात = lock + गट्टा.

11. “App हळू आहे” ची तपासणी कुठून सुरू कराल? (6.2, 7.7)

मोजून: second कुठे जातो? बहुधा रस्त्यावर (फेरे, अंतर, वजन) किंवा विना-index query तः server मोठा करणं हे बहुधा चुकीचं आणि महाग उत्तर.

12. Viral होण्याआधी कुठल्या तीन गोष्टी तयार हव्यात? (7.1, 7.6, 7.8)

Stateless servers (आडवं वाढता यावं: “काम वाटा, आठवण एकत्र”), दारावरची शिस्त (queue/rate limit: कोसळण्याऐवजी रांग), आणि खर्चाचा गजर (bill shock चा विमा). संधी दार वाजवते तेव्हा दार तुटू नये.

शेवटचं एक काम, परीक्षेबाहेरचं: Chapter 0.5 मध्ये तुम्ही एका कागदावर लिहिलं होतं: **”मी हे यासाठी शिकतोय की _____.”** तो कागद (photo) आता उघडा. वाचा. उत्तर बदललंय का? बहुतेकांचं बदलतं: सुरुवातीला ते “tech समजावं म्हणून” असतं; आता त्यात बहुधा “दुखणं-हत्यार-साखळी” ची भाषा आली असेल. तो बदल म्हणजेच हे पुस्तक.

आणि आता पुढचं दार. या पुस्तकाने एक machine दिली जी **सांगितलेलं** करते: exact recipe, exact निकाल. पण तुम्ही रोज एका machine शी बोलता जी **अंदाज** करते: तुमच्या अर्धवट वाक्याचा अर्थ लावते, कविता लिहिते, code सुचवते: आणि कधीकधी ठाम आत्मविश्वासाने **चुकते**. ती machine आतून काय आहे? Recipe कुठे आहे तिच्यात: आणि लिहिली कोणी? “शिकणं” म्हणजे machine साठी नेमकं काय? आणि या सगळ्यात पैसा कुठे वाहतो?

Book 2: The Thinking Machine. भेटू तिथे: आता तुमच्याकडे तिला उघडायची सगळी हत्यारं आहेत.

PART 5 चा शेवट: नकाशा

जे तुमच्याकडे आता आहे (Part 5)

SCALING	उभं (मोठी machine: भिंत) vs आडवं (अनेक + load balancer); मंत्र: काम वाटा, आठवण एकत्र
UPTIME	मरणं गृहीत धरा: प्रती + मरण-ओळख + failover; nines ची मोजपट्टी, प्रत्येक nine दहापट महाग
FRONT / BACK	dining hall vs kitchen; frontend वर कधीही विश्वास नाही: खरी तपासणी kitchen मध्ये
API	program चं menu card; धंद्याचा दरवाजा; वापरणारा भाडेकरू, देणारा जमीनमालक
CLOUD	दुसऱ्याच्या building मधल्या machines, भाड्याने; लवचीकपणा हेच मोल, स्वस्ताई नाही
BOTTLENECK	साखळी = कमजोर दुवा; मीटर लावा; रुंद करा / रांग लावा / दार आवळा; cascade + circuit breaker
LATENCY	second रस्त्यावर जातो: अंतर मारा (CDN) + फेरे मारा + वजन घटवा
खर्च	bill च्या तीन ओळी: compute + storage + egress; unit economics; bill shock चा गजर
दावा-तपासणी जोडणी	पाच-प्रश्नांची चौकट: कुठल्याही धंद्याचा X-ray 60 seconds ची trace; सोपं-मध्यम-अवघड नकाशा; सात-प्रश्नी spec; blameless postmortem; जिंकण्याचे 5 नियम (जाळं, दरवाजा, खिळे, वेळ, वाटप)

Book 1 ची परीक्षा मागच्या chapter मध्ये (8.6) झाली. पुढची दोन पानं संदर्भासाठी आहेत: **पूर्ण पुस्तकाचा नकाशा** (भिंतीवर लावण्यासाठी) आणि **master glossary** (पाचही parts चे शब्द एका जागी). पुस्तक वाचून झाल्यावरही ही दोन पानं तुमच्या टेबलावर जगतील: असा हेतू आहे.

PART 5 चा MINI-GLOSSARY

API	programs चं ठरलेलं प्रश्नोत्तर-menu (7.4)
bottleneck	साखळीतला सगळ्यात अरुंद दुवा (7.6)
cascade	हळा भाग -> retries -> कोसळत जाणं (7.6)
CDN	जगभरच्या godowns मधल्या प्रती (7.7)
circuit breaker	आजारी भागाला श्वास: विचारणं थांबवणं (7.6)
cloud / region	भाड्याच्या machines / त्यांचं शहर-घर (7.5)
compute	CPU-वेळेचं भाडं; bill ची ओळ 1 (7.8)
egress	बाहेर जाणाऱ्या data ची जकात; ओळ 3, कपटी (7.8)
elastic	गरजेप्रमाणे ताणणारं-आकसणारं भाडं (7.5)
failover	मुख्य मेला की प्रत आपोआप पुढे (7.2)
frontend/backend	दिसणारा भाग / kitchen; विश्वास फक्त kitchen वर (7.3)
full-stack	दोन्ही बाजू करणारा developer (7.3)
health check	"जिवंत आहेस?" ची नियमित विचारणा (7.2)
load balancer	दारावरचा वाटप करणारा (7.1)
load test	नकली गर्दीने आधीच सराव (7.6)
lock-in	एका मालकात खोल रुतणं; घर बदलणं महाग (7.5)
metrics	प्रत्येक भागावरचे मीटर (7.6)
network effect	प्रत्येक नवा user जुन्यांची किंमत वाढवतो (8.5)
nines / SLA	उपलब्धतेची मोजपट्टी / तिचं लिखित वचन (7.2)
postmortem	तुटल्यावर व्यवस्थेचं (blameless) विच्छेदन (8.4)
rate limit	दार आवळणं: प्रति-user मर्यादा (7.6)
rolling deploy	एक-एक counter बदलणं, दुकान चालू ठेवून (7.2)
spec	नेमकं-काय-हवं ची सात-प्रश्नी मागणी (8.3)
stateless	server रिकामा; state सामायिक जागी (7.1)
switching cost	सोडून जाण्याची वेदना = खिळे (8.5)
unit economics	एका user मागचा खर्च vs कमाई (7.8)
vertical/horizontal	उभं वाढणं / आडवं वाढणं (7.1)

पूर्ण नकाशा: एक पान, सगळं पुस्तक

(हे पान भिंतीवर लावा. प्रत्येक ओळीमागे एक chapter आहे; अडलात की तिथे परत.)

पैशाचं जग + machine (Parts 1-2):

LEVEL 4 AI / 3 TECH / 2 BUSINESS / 1 गरज: पैसा फक्त
Level 1 वर घुसतो, वर चढतो (0.1, 0.3)
कर्माई = size x scarcity x leverage; तास नाहीत (0.2)
लपलेला थर = महाग थर (abstraction, 0.4)
transistor -> gates -> CPU: numbers वर आधीच लिहिलेली
recipe, billion पावलं/second (1.1-1.5)
recipe = numbers => एक machine, सगळी कामं (1.6)
1 byte = अक्षर; text:photo:video = 1:हजार:लाख (2.1, 2.2)
RAM = मेज (विसराळू) / storage = गोदाम (पक्कं) (2.5)
भाषांतर-शिडी: machine code -> languages -> AI (3.1-3.4)
OS = मालक: पाळ्या, भिंती, दरबान; platform-जकात (3.5)
फुकट: दरवाजा/freemium/तुम्ही-माल/open source (3.7)
bugs = जोड्यांचा स्फोट (4.1); tests = तपासणीचा code (4.2)
Git = रोजकीर्द (4.3); software = code + माहोल (4.4)
estimate चुकणारच: छोटे चालते तुकडे, MVP (4.5)

जाळ + आठवण (Parts 3-4):

client -> (n-squared -> चौक/protocol!) -> server (5.1)
IP + router-watchman (5.2); packets, TCP/UDP (5.3)
internet = नियम, वस्तू नाही; UPI = protocol (5.4)
DNS = नाव->number ची शिडी + cache (5.5)
encryption: दोन किल्ल्या; https = रस्ता सुरक्षित,
माणूस नाही (5.6); 99% data समुद्री काच (5.7)
standard बनवणारा बाजार आखतो: DPI (5.8)
database = दरबान (6.1); index = शब्दकोश-युक्ती (6.2)
आकार: खण/पिशवी/खुंटी (6.3)
मोजकी वस्तू + अनेक हात = lock + transaction (6.4)
आत्ता vs रांगेत (6.5); cache: दाखवणं शिळं चालतं,
बदलणं ताजंच (6.6)
backup(काळ) != replication(जिवंत); 3-2-1;
न तपासलेला backup = नाहीच (6.7)

जगाला serve + जोडणी (Part 5):

आडवं वाढा: काम वाटा, आठवण एकत्र (7.1)
मरणं गृहीत धरा: प्रती + failover; nines (7.2)
विश्वास फक्त kitchen वर (7.3); API = menu card (7.4)
cloud = भाड्याचं; लवचीकपणा हेच मोल (7.5)
bottleneck / रांग / दार आवळा; cascade (7.6)
second रस्त्यावर जातो: CDN + फेरे + वजन (7.7)
bill = compute + storage + egress (7.8)
जिकणं = जाळं + दरवाजा + खिळे + वेळ + वाटप (8.5)

कुठल्याही धंद्याचा X-ray (7.9):

1. रोज पुन्हा-पुन्हा काय होत? (recipe-संधी)
2. वही कुठे, कागदी की digital? (database)
3. मोजकी वस्तू कुठली? (transaction)
4. वाट कुठे बघावी लागते? (queue)
5. पैसा कुठून कुठे वाहतो? (4 levels)

MASTER GLOSSARY: पाचही parts, एका जागी

(शब्द -> एक ओळ -> chapter. Part-निहाय याद्या त्या-त्या part च्या शेवटी आहेत; ही एकत्रित, वारंवार लागणाऱ्यांची.)

abstraction	आतली गुंतागुंत झाकून सोपं handle देणं (0.4)
API	programs चं ठरलेलं प्रश्नोत्तर-menu (7.4)
backend	kitchen: नियम, हिशोब, खरी तपासणी (7.3)
backup / restore	काळ-प्रत / परत आणणं (न तपासलेली = नाहीच) (6.7)
bit / byte	switch ची स्थिती / 8 bits = ~1 अक्षर (1.4)
bottleneck	साखळीतला अरुंद दुवा (7.6)
bug / edge case	लिहिलं-हवं ची फट / कोपऱ्यातली परिस्थिती (4.1)
cache	महाग उत्तर जवळ; किंमत: शिळेपणा (6.6)
CDN	जगभर प्रती: अंतर मारणं (7.7)
client / server	मागणारी / देणारी machine (5.1)
cloud	भाड्याच्या machines; लवचीकपणा हेच मोल (7.5)
compiler/interpreter	भाषांतर आधी / सोबत-सोबत (3.3)
CPU	recipe चालवणारा; fetch-decode-execute (1.3, 2.4)
database	data चा दरबान-server (6.1)
DNS / domain	नाव->number शिडी / website चं नाव (5.5)
deploy / DevOps	code खऱ्या जगात नेणं / त्याची व्यवस्था (4.4)
egress	बाहेर जाणाऱ्या data ची जकात (7.8)
encryption	गणिती लिफाफा; public + private किल्ली (5.6)
environment	code भोवतीचा माहोल (4.4)
failover	प्रत आपोआप पुढे येणं (7.2)
frontend	दिसणारा भाग; विश्वास ठेवायचा नाही (7.3)
Git / commit	बदलांची रोजकीर्द / एक नोंद-photo (4.3)
HTTP / HTTPS	web चे नियम / सुरक्षित रूप (5.4, 5.6)
index / query	लावलेली यादी / विचारलेला प्रश्न (6.2)
IP address	machine चा number-पत्ता (5.2)
latency	फेऱ्याचा वेळ; bandwidth पेक्षा वेगळा (2.3, 5.3)
leverage	काम एकदा, फायदा पुन्हा-पुन्हा (0.2)

load balancer	दारावरचा वाटप करणारा (7.1)
lock / race	कुंडी / नशिबावरची शर्यत (6.4)
loop / variable	पुन्हा-करा / नावाचा खण (3.2)
MVP	पहिलं छोटं चालतं रूप (4.5)
network effect	नवा user जुन्यांची किंमत वाढवतो (8.5)
nines / SLA	उपलब्धतेची मोजपट्टी / लिखित वचन (7.2)
n-squared	जोड्या वर्गाने वाढतात; protocol ने मरतो (5.1)
open source	खुला code; सामायिक हत्यार (3.7)
OS / platform	machine चा मालक / दरवाजा + जकात (3.5)
packet / TCP/UDP	message चा तुकडा / बरोबरपणा vs वेग (5.3)
pixel / sample	photo चा ठिपका / आवाजाचा क्षण (2.2)
process / state	जिवंत copy / तिची आत्ताची अवस्था (3.6)
program / code	machine साठी लिहिलेली recipe (1.3, 3.2)
protocol/standard	आधीच ठरलेले नियम / सर्वमान्य रूप (5.4, 5.8)
queue / async	कामांची रांग / नोंदवा-आत्ता, करा-नंतर (6.5)
RAM / storage	मेज (विसराळू) / गोदाम (पक्कं) (2.5)
rate limit	दार आवळणं (7.6)
replication	जिवंत प्रत; चूकही copy करते (6.7)
rollback	अर्धा गट्टा उलटा फिरवणं (6.4)
scarcity	किती कमी लोकांना जमतं (0.2)
spec	सात-प्रश्नी नेमकी मागणी (8.3)
SQL / table	tables ची भाषा / खण-आकार (6.3)
stack	थरांचा मनोरा: कोण कोणावर (0.3, 3.4)
stateless	server रिकामा; state सामायिक जागी (7.1)
transaction	सगळं-किंवा-काहीच-नाही गट्टा (6.4)
transistor	विजेने दाबला जाणारा switch (1.1)
unit economics	user-मागचा खर्च vs कमाई (7.8)
universal machine	कुठलीही recipe चालवणारी एकच machine (1.6)
version control	बदलांच्या इतिहासाची व्यवस्था (4.3)

पुस्तक संपलं. Machine आता जादू नाही. भेटू Book 2 मध्ये: The Thinking Machine.